



Universidad Nacional Autónoma De Nicaragua Managua

Universitario General de Chontales

“Cornelio Silva Argüello”

UNAN Managua, CUR Chontales



UNIVERSIDAD
NACIONAL
AUTÓNOMA DE
NICARAGUA,
MANAGUA
UNAN-MANAGUA

“2026: Por una Educación al Servicio del Pueblo”

Grupo 3

“Desarrollo de un Sistema de gestión de escritorio para el centro de estudio
Leopoldina Castrillo ”

Estudiantes

- ❖ Dhiosmer José Cuadra Miranda
- ❖ Soraya Lidieth Calero Calero
- ❖ Ana Gabriela Irias Murillo

Docente

- Jonathan Moreno
- Yesenia Tellez
- Isamar Bonilla
- Maria Rios
- Carlos Matamoro

Mayo, 2026

¡Universidad del Pueblo y para el Pueblo!





Universidad Nacional Autónoma De Nicaragua Managua

Centro Universitario General de Chontales

“Cornelio Silva Argüello”

UNAN Managua, CUR Chontales



UNIVERSIDAD
NACIONAL
AUTÓNOMA DE
NICARAGUA,
MANAGUA
UNAN-MANAGUA

“2026: Por una Educación al Servicio del Pueblo”

Grupo 3

“Desarrollo de un Sistema de gestión de escritorio para el centro de estudio
Leopoldina Castrillo ”

Estudiantes

- ❖ Dhiosmer José Cuadra Miranda(0009-0003-1054-4650)
- ❖ Soraya Lidieth Calero Calero(0009-0009-7982-7564)
- ❖ Ana Gabriela Irias Murillo(0009-0002-6150-5806)

Docente

- Jonathan Moreno
- Yesenia Tellez
- Isamar Bonilla
- Maria Rios
- Carlos Matamoro

Mayo, 2026

¡Universidad del Pueblo y para el Pueblo!





Indice

Tema.....	4
OBJETIVOS.....	5
Objetivo general.....	5
Objetivos específicos.....	5
Introducción.....	6
Descripción de la problemática.....	7
Tabla de usuarios y funcionalidades.....	8
Tabla de Roles.....	9
Historias de Usuario.....	10
Product backlog.....	13
Product Backlog priorizado.....	14
Sprint Backlog.....	14
Sprint Backlog 2.....	16
Modelo físico de la base de datos.....	19
Instrumento de levantamiento de información.....	19
Análisis estadístico descriptivo de resultados.....	19
Diseño de prototipo del sistema.....	20
Referencia Bibliográficas.....	20
Anexos.....	21
Cronograma de actividades.....	21
Tabla de asistencia.....	22





Tema

Desarrollo de una Sistema de gestión de escritorio para el centro de estudio Leopoldina
Castrillo





1. OBJETIVOS

1.1 Objetivo general

Desarrollar una aplicación de escritorio para el centro de estudio Leopoldina Castrillo que registre datos del estudiante, datos del docente, matrícula, notas, cortes evaluativos y asistencias

1.2 Objetivos específicos

- **Modelar** el sistema utilizando el paradigma **POO**, definiendo las clases y métodos necesarios para gestionar los roles de Administrador, Secretaria, Docente y Estudiante, asegurando la integridad de la información
- **Desarrollar** la aplicación de escritorio con conectividad Wi-Fi local para el uso administrativo
- **Evaluar** el impacto del sistema mediante el análisis estadístico de los tiempos de respuesta y la precisión de los reportes generados, asegurando que la herramienta contribuya a la calidad educativa del centro.





2. Introducción

El desarrollo de la aplicación para el colegio Leopoldina Castrillo Morales, en la ciudad de Juigalpa, Chontales, se llevará a cabo mediante el uso de una aplicación de escritorio donde se tendrá acceso disponible.

Ya que en la actualidad el registro de asistencias, notas de estudiantes y matrículas es uno de los principales procesos que se lleva a cabo día a día, tradicionalmente este registro se realiza de manera manual, lo que tiende a tener errores continuamente y esto reduce la entrega de notas y estabilidad de los estudiantes.

Este sistema busca modernizar la gestión interna, mejorar la calidad educativa a través de la supervisión y optimizar el tiempo de respuesta institucional. Donde se tendrá acceso el administrador y secretaria, cada uno desempeñará un rol habilitando acceso a estudiante y docente además este ofrecerá: Infraestructura de seguimiento y desarrollo de una plataforma web integral para la gestión escolar, accesible vía internet, diseñada para centralizar el control académico y administrativo. Permitirá a docentes registrar actividades y a administradores/secretarias monitorear en tiempo real, garantizando transparencia, visibilidad total del rendimiento docente y una toma de decisiones oportuna.



3. Descripción de la problemática

El centro de Estudio Leopoldina Castrillo enfrenta serias dificultades en la gestión de sus operaciones académicas y administrativas debido a que todos sus procesos se realizan de forma manual. La información relacionada con notas, asistencias, matrículas y datos de los estudiantes se registra únicamente en papel, lo cual genera desorden y dificulta el control adecuado de las actividades. Esta práctica tradicional no se adapta a las necesidades actuales, donde la eficiencia y la agilidad son fundamentales para brindar un servicio educativo confiable y de calidad.

El uso de registros manuales ha provocado la pérdida frecuente de notas y documentos importantes, el registro incorrecto de asistencias, la pérdida de matrículas y una gran dificultad para actualizar o modificar los datos de los estudiantes cuando es necesario. Además, no existe un mecanismo estructurado que permita acceder rápidamente al historial académico de los alumnos o a su información personal actualizada. Esta falta de organización genera confusión interna, duplicidad de tareas y una menor satisfacción por parte de los padres de familia y estudiantes.

Por lo tanto, se vuelve indispensable desarrollar e implementar un sistema de escritorio que sustituya los registros manuales por un entorno digital estructurado, confiable y fácil de usar. Este sistema permitirá almacenar información de estudiantes, matrículas, notas y asistencias; reducir errores operativos; agilizar la atención a padres y apoderados; y facilitar la actualización de datos sin las demoras actuales. Con esta solución tecnológica, la unidad educativa podrá optimizar su gestión interna, brindar un mejor servicio y fortalecer su posicionamiento en la comunidad.

En resumen, este sistema de escritorio permitirá optimizar procesos internos, reducir errores derivados de registros manuales, mejorar la trazabilidad de la información y ofrecer un servicio más ágil y profesional para los maestros contribuyendo al crecimiento ordenado y sostenible para los docentes.





Se concluye que la implementación de este sistema no es solo una mejora tecnológica si no una inversión estratégica que permitirá a la escuela Leopoldina castrillo optimizar sus recursos, mejorando drásticamente la experiencia de los docentes.

3.1 MARCO TEÓRICO

Sistemas de Gestión Académica

Un Sistema de Gestión Académica (SGA) es una herramienta tecnológica que permite administrar los procesos educativos de una institución. Según Álvarez (2015), estos sistemas centralizan la información de estudiantes, docentes, calificaciones, asistencias y matrículas, mejorando la eficiencia operativa.

En el Centro de Estudio Leopoldina Castrillo, la gestión manual ha generado pérdida de documentos, errores en registros y dificultades para actualizar datos. Un SGA digital permitiría superar estas limitaciones, agilizando los procesos y reduciendo errores.

3.2 Programación Orientada a Objetos (POO)

La POO es un paradigma que organiza el software en "objetos" con datos y comportamientos. Sus pilares son:

Encapsulamiento: Oculta detalles internos.

Herencia: Reutiliza código entre clases.

Polimorfismo: Un mismo método se comporta diferente según el objeto.

Abstracción: Representa lo esencial sin detalles innecesarios.

Para el sistema, se modelan clases como Estudiante, Docente, Matrícula, Asistencia y Calificación, facilitando el mantenimiento y la escalabilidad.





3.3 Patrón MVC (Modelo-Vista-Controlador)

MVC es una arquitectura que separa:

Componente	Función
------------	---------

Modelo	Datos y lógica de negocio
--------	---------------------------

Vista	Interfaz de usuario
-------	---------------------

Controlador	Intermediario entre Modelo y Vista
-------------	------------------------------------

Esta separación permite un desarrollo ordenado, facilita el mantenimiento y las pruebas. García (2010) afirma que MVC contribuye a aplicaciones más robustas y adaptables.

Java y JDBC

Java es un lenguaje orientado a objetos, multiplataforma y seguro, ideal para aplicaciones de escritorio.

JDBC (Java Database Connectivity) es la API que permite conectar Java con bases de datos relacionales, ejecutar consultas SQL y manejar transacciones.

3.4 MySQL

MySQL es un sistema de gestión de bases de datos relacional de código abierto. Ofrece alto rendimiento, escalabilidad y seguridad. Almacena los datos del sistema en tablas relacionadas, permitiendo consultas rápidas y reportes precisos.





3.5 Metodología Scrum

Scrum es un marco ágil para desarrollo de software basado en:

Sprints: Iteraciones cortas (2-4 semanas).

Roles: Product Owner, Scrum Master y Development Team.

Artefactos: Product Backlog, Sprint Backlog e Incremento.

Permite adaptarse a cambios, recibir retroalimentación continua y entregar funcionalidades de forma temprana.

3.6 MARCO CONCEPTUAL

Término	Definición
---------	------------

Administrador Usuario	con permisos para gestionar estudiantes, docentes, matrículas y reportes.
-----------------------	---

Asistencia	Registro de presencia o ausencia de un estudiante en clase.
------------	---

Calificación	Nota asignada para evaluar el rendimiento académico.
--------------	--

CRUD	Crear, Leer, Actualizar, Eliminar. Operaciones básicas de gestión de datos.
------	---

Docente	Profesional que imparte clases y evalúa a los estudiantes.
---------	--

Estudiante	Persona matriculada en una institución educativa.
------------	---

JDBC	API de Java para conectar con bases de datos.
------	---

Java	Lenguaje de programación orientado a objetos y multiplataforma.
------	---

Matrícula	Inscripción formal de un estudiante en un programa académico.
-----------	---

MVC	Patrón que separa Modelo, Vista y Controlador.
-----	--





MySQL Sistema de gestión de bases de datos relacional.

POO Paradigma que organiza el software en objetos.

Reporte Documento con información organizada del sistema.

Scrum Marco ágil para desarrollo de software.

Sistema de Gestión Académica Plataforma para administrar procesos educativos.

SQL Lenguaje para consultar y gestionar bases de datos.

Validación Verificación de que los datos ingresados sean correctos.

4. Tabla de usuarios y funcionalidades

No.	Tipo de usuario	Funcionalidad	Descripción	Encargado
1	Administrador	Registra el perfil del estudiante y docente, puede modificar los datos de cada perfil	Registrar perfil del estudiante	Dhiosmer Cuadra
2	Administrador/ secretaria	almacena el registro de calificaciones de los estudiantes	Gestión de calificaciones: ingreso, edición y consulta.	Dhiosmer Cuadra
3	Administrador	Registro de matriculas	Inscripciones estudiantiles	Soraya calero
4	Administrador	Estadísticas generales	Generación de reportes cuantitativos de los estudiantes y docentes.	Dhiosmer Cuadra



5	Docentes	Registrar Asistencias	Lleva el control diario de la presencia y ausencia del estudiante junto con los reportes de justificación de faltas o tardanza	Soraya calero
6	Docentes	Registrar notas del estudiante	Ingresa calificaciones de evaluación y tareas entre otros y mantiene un registro actualizado del rendimiento de los estudiantes	Ana Murillo

4.1 Tabla de Roles

Rol	Nombres y apellidos	Funciones
Product Owner (Propietario del producto)	Ana Irias	Es entender la necesidades de la institución Leopoldina Castrillo y definir el producto.
Scrum Master (Especialista de Scrum)	Dhiosmer Cuadra	Guiar al equipo a comprender el producto y así como motivar al equipo para entregar en tiempo y forma
Development team (Equipo de desarrollo)	Dhiosmer Cuadra	Diseñar interfaz del producto
	Soraya Calero	Diseñar interfaz de administrador, Docente y Estudiante
	Ana Irias	Pruebas del producto
	Dhiosmer Cuadra	Desarrollar la lógica de programación
Stakeholders	Leopoldina Castrillo	Institucion que usara la app



4.2 Historias de Usuario

HU01- Registrar perfil del estudiante			
Como	Administración		
Quiero	Registrar el perfil de los estudiantes		
Para	llevar un control minucioso		
Validación	❖ Se rellenaran los datos obligatorios ❖ Tendrá validación de campos por cualquier dato mas escrito ❖ Se a registro correctamente	Impacto	5
		Prioridad	5
		Estimado	2 horas

HU02- Registrar perfil del docente			
Como	Administración		
Quiero	Registrar el perfil de los docentes y profesión		
Para	Tener un mejor control de los docentes		
Validación	❖ Se rellenaran los datos obligatorios ❖ Se validaron los campos obligatorios por cualquier dato mas escrito ❖ Se a registro correctamente	Impacto	5
		Prioridad	3
		Estimado	4 horas

HU03- Editar perfil del docente y estudiante			
Como	Administración		
Quiero	Modificar el perfil del docente y estudiante		
Para	Evitar datos personales mal escritos		
Validación	❖ Se abrirá un apartado donde se podra editar los datos obligatorios ❖ Se cargan los datos ya editados al sistema	Impacto	5
		Prioridad	4
		Estimado	8 horas

HU04- Almacenar el registro de calificaciones	
Como	Administración
Quiero	Almacenar las calificaciones de cada estudiante por orden de letras
Para	llevar un control minucioso



Validación	❖ Se cargan las calificaciones proporcionada por los docentes ❖ Se validara si estan correctas ❖ Se a registro correctamente	Impacto	4
		Prioridad	2
		Estimado	8 horas

HU05-Registro de matriculas			
Como	Administración		
Quiero	Llevar el control de las matriculas que los docentes y secretaria lleve acabo		
Para	Tener un control minucioso y claro de cada estudiante		
Validación	❖ Se desplegara un formato a rellenar con los datos del estudiante y padres ❖ Se validaran los campos obligatorio ❖ Se a matriculado exitosamente	Impacto	5
		Prioridad	6
		Estimado	12 horas

HU06- Control estadístico			
Como	Administrador		
Quiero	Tener un control estadístico de matriculas, asistencias, calificaciones etc		
Para	Llevar un control mas claro de las calificaciones y asistencias de cada estudiante y docente		
Validación	❖ Se ara un control estadistico de cada apartado para ver el crecimiento del centro ❖ Se obtendran datos del estudiante y calificaciones ❖ Se dara un porcentaje por cada apartado	Impacto	5
		Prioridad	3
		Estimado	15 horas

HU07-Registrar asistencia	
Como	Docente
Quiero	Registrar un control diario de la asistencia del estudiante ausencia y reportes



Para	Tener una mejor administración por estudiante		
Validación	❖ Se asigna un formato diario por cada maestro ❖ Se rellenara según la asitencia del estudiante ❖ Se ara un reporte automático despues de varias inasistencias	Impacto	4
		Prioridad	6
		Estimado	8 horas

HU08-Registrar calificaciones			
Como	Docente		
Quiero	Registrar las calificaciones de cada estudiante		
Para	Fomentar al estudiante a tener un mejor rendimiento académico		
Validación	❖ Se entregara un formato semestral ❖ se rellenara segun las tareas, exámenes o trabajos que entreguen ❖ Se sacara el promedio y registrara	Impacto	5
		Prioridad	6
		Estimado	6 horas

4.3 Product backlog

ID	Historia de Usuario	Prioridad	impacto	Importancia	Tiempo estimado
HU001	Registrar perfil del estudiante	5	5	30	2 horas
HU002	Registrar perfil del docente	3	5	15	4 horas
HU003	Editar perfil del docente y estudiante	4	5	28	6 horas
HU004	Almacenar el registro de calificaciones	2	4	8	5 horas



HU00 5	Registro de matrículas	3	5	15	4 horas
HU00 6	Control estadístico	6	5	60	9 horas
HU00 7	Registrar asistencia	6	4	48	4 horas
HU00 8	Registrar calificaciones	6	5	30	8 horas

4.4 Product Backlog priorizado

ID	Historia de Usuario	Importancia	Sprint Programado	Fecha de Inicio	Fecha Fin
1	HU05-Registro de matriculas	360	1	20/03/2026	01/04/2026
2	HU06- Control estadístico	225	1	01/04/2026	04/04/2026
3	HU07- Registrar asistencia	192	1	04/04/2026	07/04/2026
4	HU08- Registrar calificaciones	180	1	07/04/2026	09/04/2026
5	HU03- Editar perfil del docente y estudiante	160	2	09/04/2026	15/04/2026
6	HU04- Almacenar el registro de calificaciones	64	2	15/04/2026	21/04/2026



7	HU02-Registrar perfil del docente	60	2	21/04/2026	25/04/2026
8	HU01-Registrar perfil del estudiante	50	2	25/04/2026	29/05/2026

4.5 Sprint Backlog

ID	Historia de Usuario	Tareas	Tiempo	Responsable
HU005	Registro de matriculas	Union de base de datos	6 hrs	Dhiosmer cuadra
		Diseñar interfaz de matricula	8 hrs	Ana Irias
		Funciones y validaciones de los botones guardar, cancelar, imprimir.	10 hrs	Dhiosmer Cuadra
		Probar funcionalidad	2 hrs	Soraya calero
HU006	Control estadístico	Union de base de datos	7 hrs	Dhiosmer cuadra
		Diseñar interfaz de estadistica	6 hrs	Soraya Calero



		Programar las formulas de los porcentajes y graficos	10 hrs	Dhiosmer Cuadra
		Prueba de funcionalidad	5 hrs	Ana Irias
HU007	Registrar asistencia	Union de base de datos	8 hrs	Dhiosmer Cuadra
		Diseñar interfaz del formato	5 hrs	Soraya Calero
		Programar el autoanálisis de rellenado y boton de registrar	3 hrs	Dhiosmer Cuadra
		Probar sus funciones	1 hr	Dhiosmer Cuadra
HU008	Registrar calificaciones	Union de base de datos	8 Hrs	Dhiosmer cuadra
		Diseñar la interfaz del formato de calificaciones	19 hrs	Ana Irias





		Programar las casillas para que den el porcentaje automatico tanto cualitativo como cuantitativo, buscador, boton de editar, registrar	7 hrs	Dhiosmer cuadra
		Probar sus funciones	1 hr	Soraya Calero

4.6 Sprint Backlog 2

ID	Historia de Usuario	Tareas	Tiempo	Responsable
HU003	Editar perfil del docente y estudiante	Union de base de datos	6 hrs	Dhiosmer cuadra
		Diseñar interfaz de los perfiles de edicion	8 hrs	Ana Irias
		Funciones y validaciones de los botones editar,guardar, cancelar	10 hrs	Soraya calero
		Probar funcionalidad	2 hrs	Soraya calero
HU004	Almacenar el registro de calificaciones	Union de base de datos	7 hrs	Dhiosmer cuadra



		Diseñar interfaz de los formatos ya registrados	3 hrs	Soraya Calero
		Programar los botones de imprimir, editar, buscar y guardar	15 hrs	Dhiosmer Cuadra
		Prueba de funcionalidad	5 hrs	Ana Irias
HU002	Registrar perfil del docente	Union de base de datos	4 hrs	Dhiosmer Cuadra
		Diseñar interfaz del perfil y sus campos	5 hrs	Soraya Calero
		Programar los campos obligatorios y validaciones, los botones de registrar, buscar	8 hrs	Dhiosmer Cuadra
		Probar sus funciones	2 hr	Ana Irias
HU001	Registrar perfil del estudiantes	Union de base de datos	5 Hrs	Dhiosmer cuadra

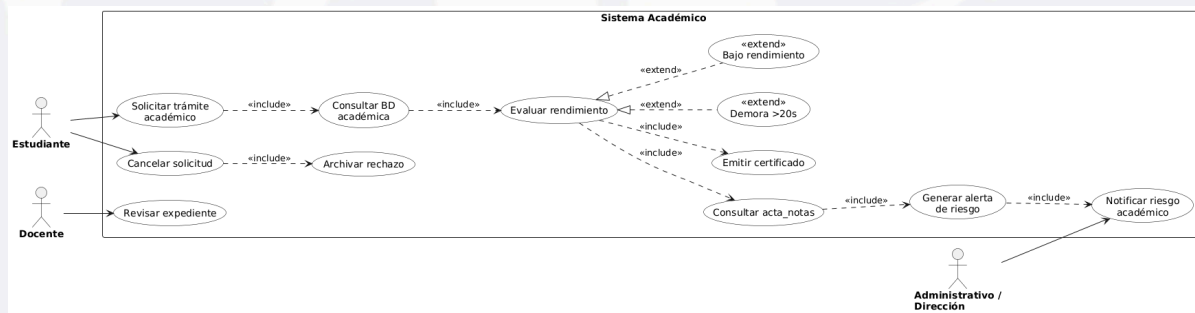
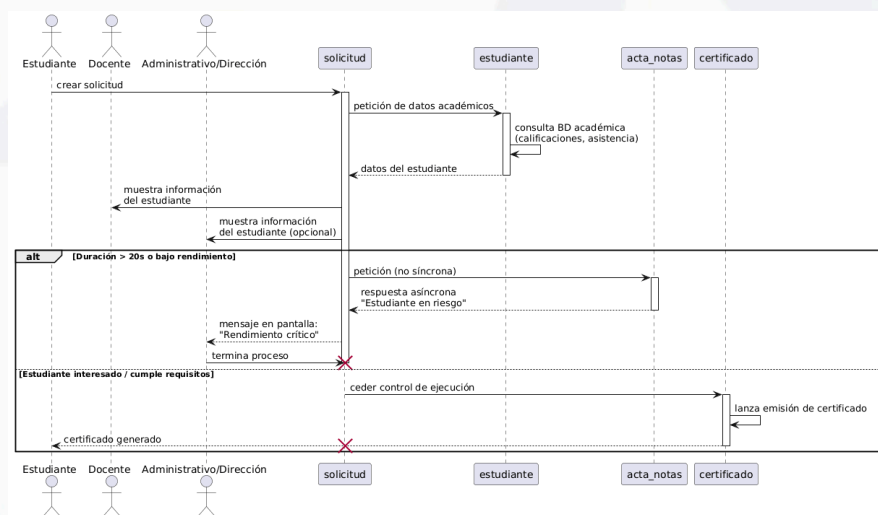
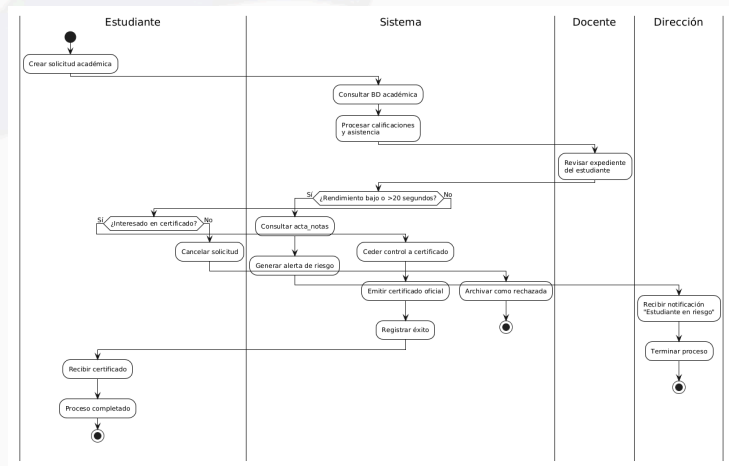


		Diseñar la interfaz del perfil del estudiante	19 hrs	Ana Irias
		Programar los campos obligatorios, verificación, buscador, registrar, editar	7 hrs	Dhiosmer cuadra
		Probar sus funciones	1 hr	Soraya Calero

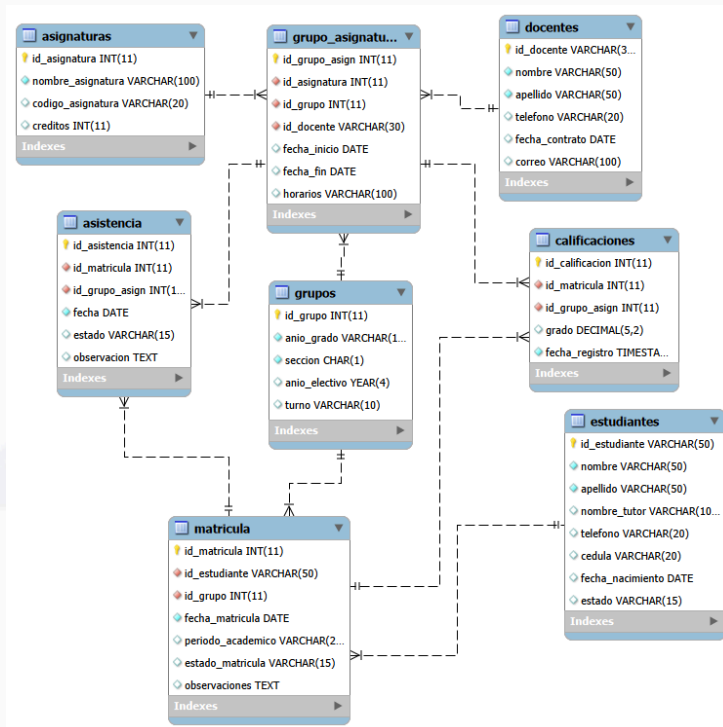


5 Diseño del sistema

5.1 Diagramas Uml



6. Modelo físico de la base de datos



6.1 Datos ingresados por tabla

```
1 • insert into Asignaturas (id_asignatura, nombre_asignatura, codigo_asignatura, credits) values
2   (1, 'Matemáticas', 'MAT-101', 4);
3
4 • INSERT INTO Docentes (id_docente, nombre, apellido, telefono, fecha_contrato, correo)
5   VALUES ('Dc-2024-5', 'David', 'Rojas', '75485230', '2024-05-15', 'Davidrojas@gmail.com');
6
7 • INSERT INTO Grupos (anio_grado, seccion, anio_electivo, turno)
8   VALUES ('1to Año', 'A', 2026, 'Mañana');
9
10 • INSERT INTO Estudiantes (id_estudiante, nombre, apellido, nombre_tutor, telefono, cedula, fecha_nacimiento, estado)
11   VALUES ('CB80-20081001-112323', 'Crismery', 'Bermudez', 'Miguel Bermudez', '75367289', '121-300889-1023Y', '2008-10-
12
13 • INSERT INTO Matricula (id_estudiante, id_grupo, fecha_matricula, periodo_academico, estado_matricula, observaciones)
14   VALUES ('AMGM-20090822-324351', 8, '2026-01-30', '2024-2', 'Activa', Null),
15   ('CJFJ-20071231-1823', 10, '2026-01-28', '2025-2', 'Activa', 'Reingreso');
16
```



```
17 • INSERT INTO Grupo_Asignatura (id_asignatura, id_grupo, id_docente, fecha_inicio, fecha_fin, horarios)
18 VALUES (1, 9, 'Dc-2001-2', '2026-02-08', '2026-11-25', 'Lun/Mie 12:40-1:30 pm'),
19 (3, 10, 'Dc-2012-2', '2026-02-15', '2026-11-16', 'Mar/Juev 2:00-2:45pm'),
20 (4, 8, 'Dc-2023-8', '2026-02-09', '2026-11-22', 'Mier/Juev 3:00-5:20pm'),
21 (5, 1, 'Dc-2019-10', '2026-02-10', '2026-11-28', 'Juev 1:00-2:45pm');
22
23 • INSERT INTO Calificaciones (id_matricula, id_grupo_asign, promedio, fecha_registro)
24 VALUES (26, 3, 60.00 , '2026-05-28 10:39:00'),
25 (29, 4, 76.00 , '2026-06-01 01:04:00'),
26 (30, 1, 100.00 , '2026-06-20 5:30:00');
27
28 • INSERT INTO Asistencia (id_matricula, id_grupo_asign, fecha, estado, observacion)
29 VALUES (25, 1, '2026-05-12', 'Presente', NULL),
30 (26, 4, '2026-05-12', 'Ausente', 'Falta irregularmente'),
31 (28, 3, '2026-05-12', 'Justificado', 'Internado en el hospital'),
32 (29, 1, '2026-05-12', 'Presente', NULL),
33 (30, 2, '2026-05-12', 'Presente', NULL);
```

6.2 Consulta por tabla con sus datos ingresados

- select * from sistemaacademico.asignaturas;
- Select * from sistemaacademicodocentes ;
- Select * from sistemaacademicogrupos;
- select * from sistemaacademicoestudiantes;
- select * from sistemaacademicomatricula;
- select * from sistemaacademicogrupo_asignatura;
- Select * from sistemaacademicocalificaciones;
- Select * from sistemaacademicoasistencia;

6.3 Asignatura

```
1 • SELECT * FROM sistemaacademico.asignaturas;
```

id_asignatura	nombre_asignatura	codigo_asignatura	creditos
1	Matemáticas	MAT-101	4
2	Geografía	GGF-104	4
3	Lengua y Literatura	LyL-102	4
4	Biología	BLG-104	3
5	Inglés	ING-105	4
6	AEP	AEP-106	1

6.4 Docentes

id_docente	nombre	apellido	telefono	fecha_contrato	correo
Dc-2001-2	Roberto	Sánchez	89291021	2026-02-20	roberto.sanchez@gmail.com
Dc-2012-2	Francisco	Villanueva	80234051	2012-02-01	Franciscovillanueva12@gmail.com
Dc-2019-10	Gloria	Gonzalez	82123212	2023-06-20	Gloriagonzalez@gmail.com
Dc-2021-1	Araselis	Cequeira	81232890	2021-01-31	Araseliscequeira@gmail.com
Dc-2023-8	Lucio	Crespo	25120156	2023-08-21	LucioCrespo@gmail.com
Dc-2024-5	David	Rojas	75485230	2024-05-15	Davidrojas@gmail.com



6.5 Grupo

	id_grupo	anio_grado	seccion	anio_electivo	turno
▶	1	1to Año	A	2026	Mañana
	2	1to Año	A	2026	Mañana
	3	1to Año	B	2026	Mañana
	4	2do Año	A	2026	Mañana
	5	2do Año	B	2026	Mañana
	6	3ro Año	A	2026	Tarde
	7	3ro Año	B	2026	Tarde

	id_grupo	anio_grado	seccion	anio_electivo	turno
	7	3ro Año	B	2026	Tarde
	8	4to Año	A	2026	Tarde
	9	4to Año	B	2026	Tarde
	10	5to Año	A	2026	Tarde
	11	5to Año	B	2026	Tarde
*	NULL	NULL	NULL	NULL	NULL

6.6 Estudiante

	id_estudiante	nombre	apellido	nombre_tutor	telefono	cedula	fecha_nacimiento	estado
▶	AMGM-20090822-324351	Ana	García	Pedro García	8295550002	121-230377-9222A	2009-08-22	Activo
	CBBO-20081001-112323	Crismary	Bermudez	Miguel Bermudez	75367289	121-300889-1023Y	2008-10-01	Activo
	CJFJ-20071231-1823	Carlos	Fonseca	Rosa Jiménez	8495550003	121-170366-1221Q	2007-12-31	Activo
	JFVM-20100515-212243	Juan	Martínez	Luisa Martínez	8095550001	121-120382-8222W	2010-05-15	Activo
	PJFM-20070524-319834	Pedro	Fernandez	Maria Martinez	89293856	121-200465-1002Y	2007-05-24	Activo
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

6.7 Matricula

	id_matricula	id_estudiante	id_grupo	fecha_matricula	periodo_academico	estado_matricula	observaciones
▶	25	CBBO-20081001-112323	3	2026-05-11	2025-2	Activa	Reingreso
	26	JFVM-20100515-212243	6	2023-08-28	2025-2	Activa	Expulsado
	28	PJFM-20070524-319834	11	2024-02-28	2020-2	Activa	Reingreso
	29	AMGM-20090822-324351	8	2026-01-30	2024-2	Activa	NULL
	30	CJFJ-20071231-1823	10	2026-01-28	2025-2	Activa	Reingreso
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

6.8 Grupo asignatura

	id_grupo_asign	id_asignatura	id_grupo	id_docente	fecha_inicio	fecha_fin	horarios
▶	1	1	9	Dc-2001-2	2026-02-08	2026-11-25	Lun/Mie 12:40-1:30 pm
	2	3	10	Dc-2012-2	2026-02-15	2026-11-16	Mar/Juev 2:00-2:45pm
	3	4	8	Dc-2023-8	2026-02-09	2026-11-22	Mier/Juev 3:00-5:20pm
	4	5	1	Dc-2019-10	2026-02-10	2026-11-28	Juev 1:00-2:45pm
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

6.9 Calificaciones

	id_calificacion	id_matricula	id_grupo_asign	promedio	fecha_registro
▶	1	25	1	70.00	2026-06-16 03:30:02
	2	28	4	80.00	2026-05-08 12:00:00
	3	26	3	60.00	2026-05-28 10:39:00
	4	29	4	76.00	2026-06-01 01:04:00
	5	30	1	100.00	2026-06-20 05:30:00
*	NULL	NULL	NULL	NULL	NULL



6.10 Asistencia

	id_asistencia	id_matricula	id_grupo_asign	fecha	estado	observacion
▶	1	25	1	2026-05-12	Presente	NULL
	2	26	4	2026-05-12	Aucnte	Falta irregularmente
	3	28	3	2026-05-12	Justificado	Internado en el hospital
	4	29	1	2026-05-12	Presente	NULL
	5	30	2	2026-05-12	Presente	NULL
*	NULL	NULL	NULL	NULL	NULL	NULL

MVC

Modelo Estudiante

```
1 |  
2 | package Modelo;  
3 |  
4 |  
5 | public class Estudiante {  
6 |     private String id_estudiante;  
7 |     private String nombre;  
8 |     private String apellido;  
9 |     private String nombre_tutor;  
10 |    private String telefono;  
11 |    private String cedula;  
12 |    private String fecha_nacimiento;  
13 |    private String estado;  
14 |  
15 |    //para guardar o eliminar  
16 |    public Estudiante(String id_estudiante, String nombre, String apellido, String nombre_tutor,  
17 |        String telefono, String cedula, String fecha_nacimiento, String estado){  
18 |        this.id_estudiante = id_estudiante;  
19 |        this.nombre = nombre;  
20 |        this.apellido = apellido;  
21 |        this.nombre_tutor = nombre_tutor;  
22 |        this.telefono = telefono;  
23 |        this.cedula = cedula;  
24 |        this.fecha_nacimiento = fecha_nacimiento;  
25 |        this.estado = estado;  
26 |    }  
27 |  
28 |    //para modificar los datos del estudiante  
29 |    public Estudiante(String id_estudiante, String nombre, String apellido, String estado, String telefono){  
30 |        this.id_estudiante = id_estudiante;  
31 |        this.nombre = nombre;  
32 |        this.apellido = apellido;  
33 |        this.estado = estado;  
34 |        this.telefono = telefono;  
35 |    }  
36 |  
37 |    public String getId_estudiante(){return id_estudiante;}  
38 |    public String getNombre(){return nombre;}  
39 |    public String getApellido(){return apellido;}  
40 |    public String getNombre_tutor(){return nombre_tutor;}  
41 |    public String getTelefono(){return telefono;}  
42 |    public String getCedula(){return cedula;}  
43 |    public String getFecha_nacimiento(){return fecha_nacimiento;}  
44 |    public String getEstado(){return estado;}  
45 |  
46 |    public void setId_estudiante(String id_estudiante){this.id_estudiante = id_estudiante;}  
47 |    public void setNombre(String nombre){this.nombre = nombre;}  
48 |    public void setApellido(String apellido){this.apellido = apellido;}  
49 |    public void setNombre_tutor(String nombre_tutor){this.nombre_tutor = nombre_tutor;}  
50 |    public void setTelefono(String telefono){this.telefono = telefono;}  
51 |    public void setCedula(String cedula){this.cedula = cedula;}  
52 |    public void setFecha_nacimiento(String fecha_nacimiento){this.fecha_nacimiento = fecha_nacimiento;}  
53 |    public void setEstado(String Estado){this.estado = Estado;}  
54 |  
55 |  
56 | }  
57 |
```



Docente

```
1 package Modelo;
2
3
4
5 public class Docentes {
6     private String id_docente;
7     private String nombre;
8     private String apellido;
9     private String telefono;
10    private String fecha_contrato;
11    private String correo;
12
13    //Guarda o eliminar
14    public Docentes (String id_docente, String nombre, String apellido,
15        String telefono, String fecha_contrato, String correo){
16        this.id_docente = id_docente;
17        this.nombre = nombre;
18        this.apellido = apellido;
19        this.telefono = telefono;
20        this.fecha_contrato = fecha_contrato;
21        this.correo = correo;
22    }
23    public Docentes (String id_docente,String nombre, String apellido,String telefono, String correo){
24        this.id_docente = id_docente;
25        this.nombre = nombre;
26        this.apellido = apellido;
27        this.telefono = telefono;
28        this.correo = correo;
29    }
30    public String getId_docente(){return id_docente;}
31    public String getNombre(){return nombre;}
32
33    public String getApellido(){return apellido;}
34    public String getTelefono(){return telefono;}
35    public String getFecha_contrato(){return fecha_contrato;}
36    public String getCorreo(){return correo;}
37
38    public void setId_docente(String id_docente){this.id_docente = id_docente;}
39    public void setNombre(String nombre){this.nombre = nombre;}
40    public void setApellido(String apellido){this.apellido = apellido;}
41    public void setTelefono(String telefono){this.telefono = telefono;}
42    public void setFecha_contrato(String fecha_contrato){this.fecha_contrato = fecha_contrato;}
43    public void setCorreo(String correo){this.correo = correo;}
44 }
45
```

Matricula

```
1 package Modelo;
2
3
4
5 public class Matriculas {
6     private int id_matricula;
7     private String id_estudiante;
8     private int id_grupo;
9     private String fecha_matricula;
10    private String periodo_academico;
11    private String estado_matricula;
12
13    //guardar o eliminar
14    public Matriculas (int id_matricula, String id_estudiante, int id_grupo,
15        String fecha_matricula, String periodo_academico, String estado_matricula){
16        this.id_matricula = id_matricula;
17        this.id_estudiante = id_estudiante;
18        this.id_grupo = id_grupo;
19        this.fecha_matricula = fecha_matricula;
20        this.periodo_academico = periodo_academico;
21        this.estado_matricula = estado_matricula;
22    }
23    //modificar
24    public Matriculas(int id_matricula, String id_estudiante, int id_grupo, String periodo_academico){
25        this.id_matricula = id_matricula;
26        this.id_estudiante = id_estudiante;
27        this.id_grupo = id_grupo;
28        this.periodo_academico = periodo_academico;
29    }
30
31    public int getId matricula(){return id matricula;}

```




```
32 public String getId_estudiante(){return id_estudiante;}
33 public int getId_grupo(){return id_grupo;}
34 public String getFecha_matricula(){return fecha_matricula;}
35 public String getPeriodo_academico(){return periodo_academico;}
36 public String getEstado_matricula(){return estado_matricula;}
37
38 public void setId_matricula(int id_matricula){this.id_matricula = id_matricula;}
39 public void setId_estudiante(String id_estudiante){this.id_estudiante = id_estudiante;}
40 public void setId_grupo(int id_grupo){this.id_grupo = id_grupo;}
41 public void setFecha_matricula(String fecha_matricula){this.fecha_matricula = fecha_matricula;}
42 public void setPeriodo_academico(String periodo_academico){this.periodo_academico = periodo_academico;}
43 public void setEstado_matricula(String estado_matricula){this.estado_matricula = estado_matricula;}
44
45 }
46
```

Calificaciones

```
1
2 package Modelo;
3
4
5 public class Calificaciones {
6     private int id_calificacion;
7     private int id_matricula;
8     private int id_grupo_asign;
9     private String promedio;
10    private String fecha_registro;
11
12    //Guardar o eliminar
13    public Calificaciones (int id_calificacion, int id_matricula,
14        int id_grupo_asign, String promedio, String fecha_registro){
15        this.id_calificacion = id_calificacion;
16        this.id_matricula = id_matricula;
17        this.id_grupo_asign = id_grupo_asign;
18        this.promedio = promedio;
19        this.fecha_registro = fecha_registro;
20    }
21    //Actualizar
22    public Calificaciones (int id_calificacion, String promedio, String fecha_registro){
23        this.id_calificacion = id_calificacion;
24        this.promedio = promedio;
25        this.fecha_registro = fecha_registro;
26    }
27    public int getId_calificacion(){return id_calificacion;}
28    public void setId_calificacion(int id_calificacion){this.id_calificacion = id_calificacion;}
29    public int getId_matricula(){return id_matricula;}
30    public void setId_matricula(int id_matricula){this.id_matricula = id_matricula;}
31    public int getId_grupo_asign(){return id_grupo_asign;}
32
33    public void setId_grupo_asign(int id_grupo_asign){this.id_grupo_asign = id_grupo_asign;}
34    public String getPromedio(){return promedio;}
35    public void setPromedio(String promedio){this.promedio = promedio;}
36    public String getFecha_registro(){return fecha_registro;}
37    public void setFecha_registro(String fecha_registro){this.fecha_registro = fecha_registro;}
38
39 }
```




Asistencia

```
2 package Modelo;
3
4
5 public class Asistencia {
6     private int id_asistencia;
7     private int id_matricula;
8     private int id_grupo_asign;
9     private String fecha;
10    private String estado;
11    private String observacion;
12
13    //guardar o eliminar
14    public Asistencia (int id_asistencia, int id_matricula, int id_grupo_asign,
15        String fecha, String estado, String observacion){
16        this.id_asistencia = id_asistencia;
17        this.id_matricula = id_matricula;
18        this.id_grupo_asign = id_grupo_asign;
19        this.fecha = fecha;
20        this.estado = estado;
21        this.observacion = observacion;
22    }
23    //actualizar
24    public Asistencia(int id_asistencia, String fecha, String estado, String observacion){
25        this.id_asistencia = id_asistencia;
26        this.fecha = fecha;
27        this.estado = estado;
28        this.observacion = observacion;
29    }
30    public int getId_asistencia(){return id_asistencia;}
31    public int getId_matricula(){return id_matricula;}
32    public int getId_grupo_asign(){return id_grupo_asign;}
33
34    public int getId_grupo_asign(){return id_grupo_asign;}
35    public String getFecha(){return fecha;}
36    public String getEstado(){return estado;}
37    public String getObservacion(){return observacion;}
38
39    public void setId_asistencia(int id_asistencia){this.id_asistencia = id_asistencia;}
40    public void setId_matricula(int id_matricula){this.id_matricula = id_matricula;}
41    public void setId_grupo_asign(int id_grupo_asign){this.id_grupo_asign = id_grupo_asign;}
42    public void setFecha(String fecha){this.fecha=fecha;}
43    public void setEstado(String estado){this.estado = estado;}
44    public void setObservacion(String observacion){this.observacion = observacion;}
45 }
```

Dao DAOEstudiante

```
1 package Modelo;
2
3 import Modelo.Estudiante;
4 import java.sql.*;
5 import java.util.ArrayList;
6 import java.util.List;
7
8 public class DAOEstudiantes {
9     private Connection conexion;
10
11    public DAOEstudiantes() {
12        try {
13            Class.forName("com.mysql.cj.jdbc.Driver");
14            conexion = DriverManager.getConnection(
15                "jdbc:mysql://localhost:3306/sistemaacademico?useSSL=false&serverTimezone=UTC",
16                "root",
17                "" // Cambia por tu contraseña de MySQL
18            );
19        } catch (Exception e) {
20            System.out.println("X Error de conexión: " + e.getMessage());
21        }
22    }
23
24    // GUARDAR - CORREGIDO
25    public boolean guardar(Estudiante e) {
26        String sql = "INSERT INTO estudiantes (id_estudiante, nombre, apellido, nombre_tutor, telefono, cedula, fecha_nacimiento, estado) VALUES (?, ?, ?, ?, ?, ?, ?, ?)";
27        try (PreparedStatement ps = conexion.prepareStatement(sql)) {
28            ps.setString(1, e.getId_estudiante());
29            ps.setString(2, e.getNombre());
30            ps.setString(3, e.getApellido());
31        }
```



```
32         ps.setString(4, e.getNombre_tutor());
33         ps.setString(5, e.getTelefono());
34         ps.setString(6, e.getCedula());
35         ps.setString(7, e.getFecha_nacimiento());
36         ps.setString(8, e.getEstado());
37         return ps.executeUpdate() > 0;
38     } catch (SQLException ex) {
39         System.out.println("X Error al guardar: " + ex.getMessage());
40         return false;
41     }
42 }
43
44 // LISTAR TODOS
45 public List<Estudiante> listarTodos() {
46     List<Estudiante> lista = new ArrayList<>();
47     String sql = "SELECT * FROM estudiantes";
48     try (Statement st = conexion.createStatement();
49          ResultSet rs = st.executeQuery(sql)) {
50         while (rs.next()) {
51             Estudiante e = new Estudiante(
52                 rs.getString("id_estudiante"),
53                 rs.getString("nombre"),
54                 rs.getString("apellido"),
55                 rs.getString("nombre_tutor"),
56                 rs.getString("telefono"),
57                 rs.getString("cedula"),
58                 rs.getString("fecha_nacimiento"),
59                 rs.getString("estado")
60             );
61             lista.add(e);
62         }
63     } catch (SQLException ex) {
64         System.out.println("X Error al listar: " + ex.getMessage());
65     }
66     return lista;
67 }
68
69 // BUSCAR POR ID
70 public Estudiante buscarPorId(String id) {
71     String sql = "SELECT * FROM estudiantes WHERE id_estudiante = ?";
72     try (PreparedStatement ps = conexion.prepareStatement(sql)) {
73         ps.setString(1, id);
74         ResultSet rs = ps.executeQuery();
75         if (rs.next()) {
76             return new Estudiante(
77                 rs.getString("id_estudiante"),
78                 rs.getString("nombre"),
79                 rs.getString("apellido"),
80                 rs.getString("nombre_tutor"),
81                 rs.getString("telefono"),
82                 rs.getString("cedula"),
83                 rs.getString("fecha_nacimiento"),
84                 rs.getString("estado")
85             );
86         }
87     } catch (SQLException ex) {
88         System.out.println("X Error al buscar: " + ex.getMessage());
89     }
90     return null;
91 }
92 }
```

```
62     }
63 } catch (SQLException ex) {
64     System.out.println("X Error al listar: " + ex.getMessage());
65 }
66 return lista;
67 }
68
69 // BUSCAR POR ID
70 public Estudiante buscarPorId(String id) {
71     String sql = "SELECT * FROM estudiantes WHERE id_estudiante = ?";
72     try (PreparedStatement ps = conexion.prepareStatement(sql)) {
73         ps.setString(1, id);
74         ResultSet rs = ps.executeQuery();
75         if (rs.next()) {
76             return new Estudiante(
77                 rs.getString("id_estudiante"),
78                 rs.getString("nombre"),
79                 rs.getString("apellido"),
80                 rs.getString("nombre_tutor"),
81                 rs.getString("telefono"),
82                 rs.getString("cedula"),
83                 rs.getString("fecha_nacimiento"),
84                 rs.getString("estado")
85             );
86         }
87     } catch (SQLException ex) {
88         System.out.println("X Error al buscar: " + ex.getMessage());
89     }
90     return null;
91 }
92 }
```

```
92
93 // ACTUALIZAR - CORREGIDO
94 public boolean actualizar(Estudiante e) {
95     String sql = "UPDATE estudiantes SET nombre = ?, apellido = ?, nombre_tutor = ?, telefono = ?, cedula = ?, fecha_nacimiento = ? WHERE id_estudiante = ?";
96     try (PreparedStatement ps = conexion.prepareStatement(sql)) {
97         ps.setString(1, e.getNombre());
98         ps.setString(2, e.getApellido());
99         ps.setString(3, e.getNombre_tutor());
100         ps.setString(4, e.getTelefono());
101         ps.setString(5, e.getCedula());
102         ps.setString(6, e.getFecha_nacimiento());
103         ps.setString(7, e.getEstado());
104         ps.setString(8, e.getId_estudiante());
105         return ps.executeUpdate() > 0;
106     } catch (SQLException ex) {
107         System.out.println("X Error al actualizar: " + ex.getMessage());
108         return false;
109     }
110 }
111
112 // ELIMINAR - CORREGIDO
113 public boolean eliminar(String id) {
114     String sql = "DELETE FROM estudiantes WHERE id_estudiante = ?";
115     try (PreparedStatement ps = conexion.prepareStatement(sql)) {
116         ps.setString(1, id);
117         return ps.executeUpdate() > 0;
118     } catch (SQLException ex) {
119         System.out.println("X Error al eliminar: " + ex.getMessage());
120         return false;
121     }
122 }
```

DAODocentes

```
1 package Modelo;
2 import Modelo.Docentes;
3 import java.sql.*;
4 import java.util.ArrayList;
5 import java.util.List;
6 public class DAODocente {
7     private Connection conexion;
8
9     public DAODocente() {
10         try {
11             Class.forName("com.mysql.cj.jdbc.Driver");
12             conexion = DriverManager.getConnection(
13                 "jdbc:mysql://localhost:3306/sistemaacademico?useSSL=false&serverTimezone=UTC",
14                 "root",
15                 "" // Cambia por tu contraseña de MySQL
16             );
17         } catch (Exception e) {
18             System.out.println("X Error de conexión: " + e.getMessage());
19         }
20     }
21
22     public boolean guardar(Docentes d) {
23         String sql = "INSERT INTO docentes (id_docente, nombre, apellido, telefono, fecha_contrato, correo) VALUES (?, ?, ?, ?, ?, ?)";
24         try (PreparedStatement ps = conexion.prepareStatement(sql)) {
25             ps.setString(1, d.getId_docente());
26             ps.setString(2, d.getNombre());
27             ps.setString(3, d.getApellido());
28             ps.setString(4, d.getTelefono());
29             ps.setString(5, d.getFecha_contrato());
30             ps.setString(6, d.getCorreo());
31             ps.executeUpdate();
32         } catch (SQLException ex) {
33             System.out.println("X Error al guardar: " + ex.getMessage());
34             return false;
35         }
36     }
37
38     public List<Docentes> listarTodos() {
39         List<Docentes> lista = new ArrayList<>();
40         String sql = "SELECT * FROM docentes";
41         try (Statement st = conexion.createStatement()) {
42             ResultSet rs = st.executeQuery(sql);
43             while (rs.next()) {
44                 Docentes d = new Docentes(
45                     rs.getString("id_docente"),
46                     rs.getString("nombre"),
47                     rs.getString("apellido"),
48                     rs.getString("telefono"),
49                     rs.getString("fecha_contrato"),
50                     rs.getString("correo")
51                 );
52                 lista.add(d);
53             }
54         } catch (SQLException ex) {
55             System.out.println("X Error al listar: " + ex.getMessage());
56         }
57         return lista;
58     }
59
60     // BUSCAR POR ID
61     public Docentes buscarPorId(String id) {
62         String sql = "SELECT * FROM docentes WHERE id_docente = ?";
63     }
```



```
63 String sql = "SELECT * FROM docentes WHERE id_docente = ?";
64 try (PreparedStatement ps = conexion.prepareStatement(sql)) {
65     ps.setString(1, id);
66     ResultSet rs = ps.executeQuery();
67     if (rs.next()) {
68         return new Docentes(
69             rs.getString("id_docente"),
70             rs.getString("nombre"),
71             rs.getString("apellido"),
72             rs.getString("telefono"),
73             rs.getString("fecha_contrato"),
74             rs.getString("correo")
75         );
76     }
77 } catch (SQLException ex) {
78     System.out.println("X Error al buscar: " + ex.getMessage());
79 }
80 return null;
81 }
82
83
84
85
86 // ACTUALIZAR - CORREGIDO
87 // ACTUALIZAR - SOLO 4 CAMPOS (nombre, apellido, telefono, correo)
88 public boolean actualizar(Docentes d) {
89     String sql = "UPDATE docentes SET nombre = ?, apellido = ?, telefono = ?, correo = ? WHERE id_docente = ?";
90     try (PreparedStatement ps = conexion.prepareStatement(sql)) {
91         ps.setString(1, d.getNombre());
92         ps.setString(2, d.getApellido());
93         ps.setString(3, d.getTelefono());
94         ps.setString(4, d.getCorreo());
95         ps.setString(5, d.getId_docente());
96         return ps.executeUpdate() > 0;
97     } catch (SQLException ex) {
98         System.out.println("X Error al actualizar: " + ex.getMessage());
99         return false;
100     }
101 }
102
103 // ELIMINAR - CORREGIDO
104 public boolean eliminar(String id) {
105     try {
106         // Desactivar verificación de llaves foráneas
107         Statement st = conexion.createStatement();
108         st.execute("SET FOREIGN_KEY_CHECKS = 0");
109         st.close();
110
111         // CORREGIDO: id_docente (sin "s" al final)
112         String sql = "DELETE FROM docentes WHERE id_docente = ?";
113         PreparedStatement ps = conexion.prepareStatement(sql);
114         ps.setString(1, id);
115         int resultado = ps.executeUpdate();
116         ps.close();
117
118         // Reactivar verificación de llaves foráneas
119         st = conexion.createStatement();
120         st.execute("SET FOREIGN_KEY_CHECKS = 1");
121         st.close();
122
123         return resultado > 0;
124     } catch (SQLException ex) {
125         System.out.println("X Error al eliminar: " + ex.getMessage());
126     }
127 }
```



```
121
122         return resultado > 0;
123     } catch (SQLException ex) {
124         System.out.println("X Error al eliminar: " + ex.getMessage());
125         return false;
126     }
127 }
128 }
```

DAOMatricula

```
1 package Modelo;
2
3
4 import Modelo.Matriculas;
5 import java.sql.*;
6 import java.util.ArrayList;
7 import java.util.List;
8 public class DAOMatricula {
9     private Connection conexion;
10
11     public DAOMatricula() {
12         try {
13             Class.forName("com.mysql.cj.jdbc.Driver");
14             conexion = DriverManager.getConnection(
15                 "jdbc:mysql://localhost:3306/sistemaacademico?useSSL=false&serverTimezone=UTC",
16                 "root",
17                 "" // Cambia por tu contraseña de MySQL
18             );
19         } catch (Exception e) {
20             System.out.println("X Error de conexión: " + e.getMessage());
21         }
22     }
23
24     // GUARDAR - CORREGIDO
25     public boolean guardar(Matriculas m) {
26         String sql = "INSERT INTO matricula (id_matricula, id_estudiante, id_grupo, fecha_matricula, periodo_academico, estado_matricula) VALUES (?, ?, ?, ?, ?, ?)";
27         try (PreparedStatement ps = conexion.prepareStatement(sql)) {
28             ps.setInt(1, m.getId_matricula());
29             ps.setString(2, m.getId_estudiante());
30             ps.setInt(3, m.getId_grupo());
31             ps.setString(4, m.getFecha_matricula());
32
33             ps.setString(5, m.getPeriodo_academico());
34             ps.setString(6, m.getEstado_matricula());
35
36             return ps.executeUpdate() > 0;
37         } catch (SQLException ex) {
38             System.out.println("X Error al guardar: " + ex.getMessage());
39             return false;
40         }
41     }
42
43     // LISTAR TODOS
44     public List<Matriculas> listarTodos() {
45         List<Matriculas> lista = new ArrayList<>();
46         String sql = "SELECT * FROM matricula";
47         try (Statement st = conexion.createStatement();
48             ResultSet rs = st.executeQuery(sql)) {
49             while (rs.next()) {
50                 Matriculas m = new Matriculas(
51                     rs.getInt("id_matricula"),
52                     rs.getString("id_estudiante"),
53                     rs.getInt("id_grupo"),
54                     rs.getString("fecha_matricula"),
55                     rs.getString("periodo_academico"),
56                     rs.getString("estado_matricula")
57                 );
58                 lista.add(m);
59             }
60         } catch (SQLException ex) {
61             System.out.println("X Error al listar: " + ex.getMessage());
62         }
63     }
64 }
```



```
63         return lista;
64     }
65
66     // BUSCAR POR ID
67     public Matriculas buscarPorId(int id_matricula) {
68         String sql = "SELECT * FROM matricula WHERE id_matricula = ?";
69         try (PreparedStatement ps = conexion.prepareStatement(sql)) {
70             ps.setInt(1, id_matricula);
71             ResultSet rs = ps.executeQuery();
72             if (rs.next()) {
73                 return new Matriculas(
74                     rs.getInt("id_matricula"),
75                     rs.getString("id_estudiante"),
76                     rs.getInt("id_grupo"),
77                     rs.getString("fecha_matricula"),
78                     rs.getString("periodo_academico"),
79                     rs.getString("estado_matricula")
80                 );
81             }
82         } catch (SQLException ex) {
83             System.out.println("X Error al buscar: " + ex.getMessage());
84         }
85         return null;
86     }
87
88     // ACTUALIZAR - CORREGIDO
89     public boolean actualizar(Matriculas m) {
90         String sql = "UPDATE matricula SET id_estudiante= ?, id_grupo= ?, periodo_academico = ? WHERE id_matricula = ?";
91         try (PreparedStatement ps = conexion.prepareStatement(sql)) {
92             ps.setInt(1, m.getId_matricula());
93             ps.setString(2, m.getId_estudiante());
94             ps.setInt(3, m.getId_grupo());
95             ps.setString(4, m.getPeriodo_academico());
96
97             return ps.executeUpdate() > 0;
98         } catch (SQLException ex) {
99             System.out.println("X Error al actualizar: " + ex.getMessage());
100             return false;
101         }
102     }
103
104     // ELIMINAR - CORREGIDO
105     public boolean eliminar(String id) {
106         String sql = "DELETE FROM matricula WHERE id_matricula = ?";
107         try (PreparedStatement ps = conexion.prepareStatement(sql)) {
108             ps.setString(1, id);
109             return ps.executeUpdate() > 0;
110         } catch (SQLException ex) {
111             System.out.println("X Error al eliminar: " + ex.getMessage());
112             return false;
113         }
114     }
115 }
116
```


DAOCalificaciones

```
1 package Modelo;
2
3 import Modelo.Calificaciones;
4 import java.sql.*;
5 import java.util.ArrayList;
6 import java.util.List;
7 public class DAOCalificaciones {
8     private Connection conexion;
9
10    public DAOCalificaciones() {
11        try {
12            Class.forName("com.mysql.cj.jdbc.Driver");
13            conexion = DriverManager.getConnection(
14                "jdbc:mysql://localhost:3306/sistemaacademico?useSSL=false&serverTimezone=UTC",
15                "root",
16                "" // Cambia por tu contraseña de MySQL
17            );
18        } catch (Exception e) {
19            System.out.println("X Error de conexión: " + e.getMessage());
20        }
21    }
22
23    // GUARDAR - CORREGIDO
24    public boolean guardar(Calificaciones c) {
25        String sql = "INSERT INTO calificaciones (id_calificacion, id_matricula, id_grupo_asign, promedio, fecha_registro) VALUES
26        try (PreparedStatement ps = conexion.prepareStatement(sql)) {
27            ps.setInt(1, c.getId_calificacion());
28            ps.setInt(2, c.getId_matricula());
29            ps.setInt(3, c.getId_grupo_asign());
30            ps.setString(4, c.getPromedio());
31            ps.setString(5, c.getFecha_registro());
32
33
34            return ps.executeUpdate() > 0;
35        } catch (SQLException ex) {
36            System.out.println("X Error al guardar: " + ex.getMessage());
37            return false;
38        }
39    }
40
41    // LISTAR TODOS
42    public List<Calificaciones> listarTodos() {
43        List<Calificaciones> lista = new ArrayList<>();
44        String sql = "SELECT * FROM calificaciones";
45        try (Statement st = conexion.createStatement();
46            ResultSet rs = st.executeQuery(sql)) {
47            while (rs.next()) {
48                Calificaciones c = new Calificaciones(
49                    rs.getInt("id_calificacion"),
50                    rs.getInt("id_matricula"),
51                    rs.getInt("id_grupo_asign"),
52                    rs.getString("promedio"),
53                    rs.getString("fecha_registro")
54                );
55                lista.add(c);
56            }
57        } catch (SQLException ex) {
58            System.out.println("X Error al listar: " + ex.getMessage());
59        }
60        return lista;
61    }
62 }
63
```



```
64 // BUSCAR POR ID
65 public Calificaciones buscarPorId(int id_calificacion) {
66     String sql = "SELECT * FROM calificaciones WHERE id_calificacion = ?";
67     try (PreparedStatement ps = conexion.prepareStatement(sql)) {
68         ps.setInt(1, id_calificacion);
69         ResultSet rs = ps.executeQuery();
70         if (rs.next()) {
71             return new Calificaciones(
72                 rs.getInt("id_calificacion"),
73                 rs.getInt("id_matricula"),
74                 rs.getInt("id_grupo_asign"),
75                 rs.getString("promedio"),
76                 rs.getString("fecha_registro")
77             );
78         }
79     } catch (SQLException ex) {
80         System.out.println("X Error al buscar: " + ex.getMessage());
81     }
82     return null;
83 }
84
85 // ACTUALIZAR - CORREGIDO
86
87 // ACTUALIZAR - CORREGIDO
88 public boolean actualizar(Calificaciones c) {
89     String sql = "UPDATE calificaciones SET promedio = ?, fecha_registro = ? WHERE id_calificacion = ?";
90     try (PreparedStatement ps = conexion.prepareStatement(sql)) {
91         ps.setString(1, c.getPromedio());
92         ps.setString(2, c.getFecha_registro());
93         ps.setInt(3, c.getId_calificacion());
94
95         ps.setInt(3, c.getId_calificacion());
96
97         int resultado = ps.executeUpdate();
98         System.out.println(" Filas afectadas: " + resultado);
99         return resultado > 0;
100     } catch (SQLException ex) {
101         System.out.println(" Error al actualizar: " + ex.getMessage());
102         return false;
103     }
104 }
105
106 // ELIMINAR - CORREGIDO
107 public boolean eliminar(String id) {
108     String sql = "DELETE FROM calificaciones WHERE id_calificacion = ?";
109     try (PreparedStatement ps = conexion.prepareStatement(sql)) {
110         ps.setString(1, id);
111         return ps.executeUpdate() > 0;
112     } catch (SQLException ex) {
113         System.out.println("X Error al eliminar: " + ex.getMessage());
114         return false;
115     }
116 }
117 }
```



DAOAsistencia

```
1 package Modelo;
2 import Modelo.Asistencia;
3 import java.sql.*;
4 import java.util.ArrayList;
5 import java.util.List;
6 public class DAOAsistencia {
7     private Connection conexion;
8
9
10    public DAOAsistencia() {
11        try {
12            Class.forName("com.mysql.cj.jdbc.Driver");
13            conexion = DriverManager.getConnection(
14                "jdbc:mysql://localhost:3306/sistemaacademico?useSSL=false&serverTimezone=UTC",
15                "root",
16                "" // Cambia por tu contraseña de MySQL
17            );
18        } catch (Exception e) {
19            System.out.println("X Error de conexión: " + e.getMessage());
20        }
21    }
22
23    // GUARDAR - CORREGIDO
24    public boolean guardar(Asistencia a) {
25        String sql = "INSERT INTO asistencia (id_asistencia, id_matricula, id_grupo_asign, fecha, estado, observacion) VALUES (?, ?, ?, ?, ?, ?)";
26        try (PreparedStatement ps = conexion.prepareStatement(sql)) {
27            ps.setInt(1, a.getId_asistencia());
28            ps.setInt(2, a.getId_matricula());
29            ps.setInt(3, a.getId_grupo_asign());
30            ps.setString(4, a.getFecha());
31            ps.setString(5, a.getEstado());
32            ps.setString(6, a.getObservacion());
33
34            return ps.executeUpdate() > 0;
35        } catch (SQLException ex) {
36            System.out.println("X Error al guardar: " + ex.getMessage());
37            return false;
38        }
39    }
40
41    // LISTAR TODOS
42    public List<Asistencia> listarTodos() {
43        List<Asistencia> lista = new ArrayList<>();
44        String sql = "SELECT * FROM asistencia";
45        try (Statement st = conexion.createStatement();
46             ResultSet rs = st.executeQuery(sql)) {
47            while (rs.next()) {
48                Asistencia a = new Asistencia(
49                    rs.getInt("id_asistencia"),
50                    rs.getInt("id_matricula"),
51                    rs.getInt("id_grupo_asign"),
52                    rs.getString("fecha"),
53                    rs.getString("estado"),
54                    rs.getString("observacion")
55                );
56                lista.add(a);
57            }
58        } catch (SQLException ex) {
59            System.out.println("X Error al listar: " + ex.getMessage());
60        }
61        return lista;
62    }
63 }
```



```
66 public Asistencia buscarPorId(int id_asistencia) {
67     String sql = "SELECT * FROM asistencia WHERE id_asistencia = ?";
68     try (PreparedStatement ps = conexion.prepareStatement(sql)) {
69         ps.setInt(1, id_asistencia);
70         ResultSet rs = ps.executeQuery();
71         if (rs.next()) {
72             return new Asistencia(
73                 rs.getInt("id_asistencia"),
74                 rs.getInt("id_matricula"),
75                 rs.getInt("id_grupo_asign"),
76                 rs.getString("fecha"),
77                 rs.getString("estado"),
78                 rs.getString("observacion")
79             );
80         }
81     } catch (SQLException ex) {
82         System.out.println("X Error al buscar: " + ex.getMessage());
83     }
84     return null;
85 }
86
87 // ACTUALIZAR - CORREGIDO
88
89 // ACTUALIZAR - CORREGIDO
90 public boolean actualizar(Asistencia a) {
91     String sql = "UPDATE asistencia SET fecha = ?, estado = ?, observacion = ? WHERE id_asistencia = ?";
92     try (PreparedStatement ps = conexion.prepareStatement(sql)) {
93         ps.setString(1, a.getFecha());
94         ps.setString(2, a.getEstado());
95         ps.setString(3, a.getObservacion());
96         ps.setInt(4, a.getId_asistencia());
97
98         int resultado = ps.executeUpdate();
99         System.out.println(" Filas afectadas: " + resultado);
100         return resultado > 0;
101     } catch (SQLException ex) {
102         System.out.println(" Error al actualizar: " + ex.getMessage());
103         return false;
104     }
105 }
106
107 // ELIMINAR - CORREGIDO
108 public boolean eliminar(String id) {
109     String sql = "DELETE FROM asistencia WHERE id_asistencia = ?";
110     try (PreparedStatement ps = conexion.prepareStatement(sql)) {
111         ps.setString(1, id);
112         return ps.executeUpdate() > 0;
113     } catch (SQLException ex) {
114         System.out.println("X Error al eliminar: " + ex.getMessage());
115         return false;
116     }
117 }
118
119 }
```



Controladores

C.Estudiante

```
1 |
2 | package Controlador;
3 |
4 | import Modelo.Estudiante;
5 | import Modelo.DAOEstudiantes;
6 | import java.util.List;
7 |
8 | public class EstudianteControlador {
9 |     private DAOEstudiantes dao;
10 |
11 |     public EstudianteControlador() {
12 |         this.dao = new DAOEstudiantes();
13 |     }
14 |
15 |     public boolean guardar(String id, String nombre, String apellido, String tutor,
16 |         String telefono, String cedula, String fechaNac, String estado) {
17 |         if (id.isEmpty() || nombre.isEmpty()) {
18 |             return false;
19 |         }
20 |         Estudiante e = new Estudiante(id, nombre, apellido, tutor, telefono, cedula, fechaNac, estado);
21 |         return dao.guardar(e);
22 |     }
23 |
24 |     public List<Estudiante> obtener() {
25 |         return dao.listarTodos();
26 |     }
27 |
28 |     public boolean actualizar(String id, String nombre, String apellido, String estado, String telefono) {
29 |         Estudiante e = dao.buscarPorId(id);
30 |         if (e == null) {
31 |             return false;
32 |         }
33 |         e.setNombre(nombre);
34 |         e.setApellido(apellido);
35 |         e.setEstado(estado);
36 |         e.setTelefono(telefono);
37 |         return dao.actualizar(e);
38 |     }
39 |
40 |     public boolean eliminar(String id) {
41 |         return dao.eliminar(id);
42 |     }
43 | }
```



C.Docente

```
1 package Controlador;
2
3 import Modelo.Docentes;
4 import Modelo.DAODocente;
5 import java.util.List;
6
7 public class DocentesControlador {
8     private DAODocente dao;
9
10    public DocentesControlador() {
11        this.dao = new DAODocente();
12    }
13
14    public boolean guardar(String id, String nombre, String apellido, String telefono,
15        String fechaContrato, String correo) {
16        if (id.isEmpty() || nombre.isEmpty()) {
17            return false;
18        }
19        Docentes d = new Docentes(id, nombre, apellido, telefono, fechaContrato, correo);
20        return dao.guardar(d);
21    }
22
23    public List<Docentes> obtener() {
24        return dao.listarTodos();
25    }
26
27    public boolean actualizar(String id, String nombre, String apellido, String telefono, String correo)
28        Docentes d = dao.buscarPorId(id);
29        if (d == null) {
30            return false;
31        }
32
33        d.setNombre(nombre);
34        d.setApellido(apellido);
35        d.setTelefono(telefono);
36        d.setCorreo(correo);
37        return dao.actualizar(d);
38    }
39
40    public boolean eliminar(String id) {
41        return dao.eliminar(id);
42    }
43 }
```




C.Matricula

```
1 |
2 | package Controlador;
3 | import Modelo.Matriculas;
4 | import Modelo.DAOMatricula;
5 | import java.util.List;
6 |
7 | public class MatriculaControlador {
8 |     private DAOMatricula dao;
9 |
10 |    public MatriculaControlador() {
11 |        this.dao = new DAOMatricula();
12 |    }
13 |
14 |    public boolean guardar ( int id_matricula, String id_estudiante,
15 |        int id_grupo, String fecha_matricula, String periodo_academico, String estado_matricula ){
16 |        Matriculas m = new Matriculas(id_matricula, id_estudiante, id_grupo, fecha_matricula, periodo_ac
17 |        return dao.guardar(m);
18 |    }
19 |    public List<Matriculas> obtener() {
20 |        return dao.listarTodos();
21 |    }
22 |    public boolean actualizar(int id_matricula, String id_estudiante, int id_grupo,
23 |        String periodo_academico, String estado_matricula) {
24 |        Matriculas m = dao.buscarPorId(id_matricula);
25 |        if (m == null) {
26 |            return false;
27 |        }
28 |        m.setId_estudiante(id_estudiante);
29 |        m.setId_grupo(id_grupo);
30 |        m.setPeriodo_academico(periodo_academico);
31 |        m.setEstado_matricula(estado_matricula);
32 |        return dao.actualizar(m);
33 |    }
34 |
35 |    public boolean eliminar(String id) {
36 |        return dao.eliminar(id);
37 |    }
38 | }
39 |
```



C.Calificaciones

```
1 |
2 | package Controlador;
3 | import Modelo.Calificaciones;
4 | import Modelo.DAOAsistencia;
5 | import Modelo.DAOCalificaciones;
6 | import java.util.List;
7 | public class CalificacionesControlador {
8 |     private DAOCalificaciones dao;
9 |
10 |     public CalificacionesControlador() {
11 |         this.dao = new DAOCalificaciones();
12 |     }
13 |
14 |     public boolean guardar ( int id_calificacion, int id_matricula,
15 |         int id_grupo_asign,String promedio, String fecha_registro ){
16 |         Calificaciones c = new Calificaciones(id_calificacion, id_matricula, id_grupo_asign, promedio, fecha_registro);
17 |         return dao.guardar(c);
18 |     }
19 |     public List<Calificaciones> obtener() {
20 |         return dao.listarTodos();
21 |     }
22 |     public boolean actualizar( int id_calificacion,String promedio, String fecha_registro) {
23 |         Calificaciones c = dao.buscarPorId(id_calificacion);
24 |         if (c == null) {
25 |             return false;
26 |         }
27 |         c.setPromedio(promedio);
28 |         c.setFecha_registro(fecha_registro);
29 |
30 |         return dao.actualizar(c);
31 |     }
32 |
33 |     public boolean eliminar(String id) {
34 |         return dao.eliminar(id);
35 |     }
36 | }
37 |
38 |
```



C.Asistencia

```
1 package Controlador;
2
3 import Modelo.Asistencia;
4 import Modelo.DAOAsistencia;
5 import java.util.List;
6
7 public class AsistenciaControlador {
8     private DAOAsistencia dao;
9
10    public AsistenciaControlador() {
11        this.dao = new DAOAsistencia();
12    }
13
14    public boolean guardar ( int id_asistencia, int id_matricula,
15        int id_grupo_asign, String fecha, String estado, String observacion ){
16        Asistencia a = new Asistencia(id_asistencia, id_matricula, id_grupo_asign, fecha, estado, observacion);
17        return dao.guardar(a);
18    }
19    public List<Asistencia> obtener() {
20        return dao.listarTodos();
21    }
22    public boolean actualizar(int id_asistencia, String fecha, String estado, String observacion) {
23        Asistencia a = dao.buscarPorId(id_asistencia);
24        if (a == null) {
25            return false;
26        }
27        a.setFecha(fecha);
28        a.setEstado(estado);
29        a.setObservacion(observacion);
30        return dao.actualizar(a);
31    }
32
33    public boolean eliminar(String id) {
34        return dao.eliminar(id);
35    }
36 }
37
38
```

Vistasclases





V.Estudiante

```
1 package Vista;
2 import Controlador.EstudianteControlador;
3 import Modelo.Estudiante;
4 import java.util.List;
5 import java.util.Scanner;
6
7 public class VistaEstudiante {
8     public void MenuEstudiante() {
9         Scanner sc = new Scanner(System.in);
10        EstudianteControlador controller = new EstudianteControlador();
11        int opcion;
12
13        do {
14            System.out.println("\n--- MENU DE ESTUDIANTES");
15            System.out.println("1. Guardar estudiante");
16            System.out.println("2. Mostrar estudiantes");
17            System.out.println("3. Actualizar estudiante");
18            System.out.println("4. Eliminar estudiante");
19            System.out.println("0. Salir ");
20            System.out.print("Opcion: ");
21            opcion = sc.nextInt();
22            sc.nextLine(); // Limpiar buffer
23
24            switch (opcion) {
25                case 1:
26                    System.out.print("Id_estudiante: ");
27                    String id_estudiante = sc.nextLine();
28
29                    System.out.print("Nombre: ");
30                    String nombre = sc.nextLine();
31
32                    System.out.print("Apellido: ");
33                    String apellido = sc.nextLine();
34
35                    System.out.print("Nombre del tutor: ");
36                    String nombre_tutor = sc.nextLine();
37
38                    System.out.print("Telefono: ");
39                    String telefono = sc.nextLine();
40
41                    System.out.print("Cedula: ");
42                    String cedula = sc.nextLine();
43
44                    System.out.print("Fecha de nacimiento YYYY-MM-DD: ");
45                    String fecha_nacimiento = sc.nextLine();
46
47                    System.out.print("Estado: ");
48                    String estado = sc.nextLine();
49
50                    if (controller.guardar(id_estudiante, nombre, apellido, nombre_tutor,
51                                         telefono, cedula, fecha_nacimiento, estado)) {
52                        System.out.println("✓ Estudiante guardado");
53                    } else {
54                        System.out.println("✗ Error al guardar");
55                    }
56                    break;
57
58                    case 2:
59                        List<Estudiante> lista = controller.obtener();
60
61                        System.out.println("\n--- LISTA DE ESTUDIANTES");
62                        if (lista.isEmpty()) {
```



```
63         System.out.println("⚠ No hay estudiantes registrados");
64     } else {
65         for (Estudiante e : lista) {
66             System.out.println(e.getId_estudiante()
67                 + " - " + e.getNombre()
68                 + " - " + e.getApellido()
69                 + " - " + e.getNombre_tutor()
70                 + " - " + e.getTelefono()
71                 + " - " + e.getCedula()
72                 + " - " + e.getFecha_nacimiento()
73                 + " - " + e.getEstado());
74         }
75     }
76     break;
77
78     case 3:
79         System.out.print("ID del estudiante a modificar: ");
80         String ide = sc.nextLine();
81
82         System.out.print("Nuevo nombre: ");
83         String nombree = sc.nextLine();
84
85         System.out.print("Nuevo apellido: ");
86         String apellidoe = sc.nextLine();
87
88         System.out.print("Nuevo estado: ");
89         String estadoe = sc.nextLine();
90
91         System.out.print("Nuevo telefono: ");
92         String telefonoe = sc.nextLine();
93
94         if (controller.actualizar(ide, nombree, apellidoe, estadoe, telefonoe)) {
95             System.out.println("✔ Estudiante actualizado");
96         } else {
97             System.out.println("✗ Error al actualizar");
98         }
99         break;
100
101     case 4:
102         System.out.print("ID del estudiante a eliminar: ");
103         String id = sc.nextLine(); // CORREGIDO: int -> String
104         if (controller.eliminar(id)) {
105             System.out.println("✔ Estudiante eliminado");
106         } else {
107             System.out.println("✗ Error al eliminar");
108         }
109         break;
110
111     case 0:
112         System.out.println("🏠 Volviendo al menu principal...");
113         break;
114
115     default:
116         System.out.println("✗ Opcion no valida");
117         break;
118     }
119 }
120 } while (opcion != 0);
121 }
122 }
```

V.Docente

```
1 package Vista;
2
3
4 import Controlador.DocentesControlador;
5 import Modelo.Docentes;
6 import java.util.List;
7 import java.util.Scanner;
8
9 public class VistaDocentes {
10     public void MenuDocente() {
11         Scanner sc = new Scanner(System.in);
12         DocentesControlador controller = new DocentesControlador();
13         int opcion;
14
15         do {
16             System.out.println("\n--- MENU DE DOCENTES");
17             System.out.println("1. Guardar Docentes");
18             System.out.println("2. Mostrar Docentes");
19             System.out.println("3. Actualizar Docentes");
20             System.out.println("4. Eliminar Docentes");
21             System.out.println("0. Salir ");
22             System.out.print("Opcion: ");
23             opcion = sc.nextInt();
24             sc.nextLine(); // Limpiar buffer
25
26             switch (opcion) {
27                 case 1:
28                     System.out.print("Id_docente: ");
29                     String id_docente = sc.nextLine();
30
31                     System.out.print("Nombre: ");
32
33                     String nombre = sc.nextLine();
34
35                     System.out.print("Apellido: ");
36                     String apellido = sc.nextLine();
37
38                     System.out.print("telefono: ");
39                     String telefono = sc.nextLine();
40
41                     System.out.print("Fecha de contrato YYYY-MM-DD: ");
42                     String fecha_contrato = sc.nextLine();
43
44                     System.out.print("Correo: ");
45                     String correo = sc.nextLine();
46
47                     if (controller.guardar(id_docente, nombre, apellido, telefono, fecha_contrato, correo)) {
48                         System.out.println("✓ Docente guardado");
49                     } else {
50                         System.out.println("✗ Error al guardar");
51                     }
52                     break;
53
54                 case 2:
55                     List<Docentes> lista = controller.obtener();
56
57                     System.out.println("\n--- Lista De Docentes");
58                     if (lista.isEmpty()) {
59                         System.out.println("⚠ No hay Docentes registrados");
60                     } else {
61                         for (Docentes d : lista) {
62                             System.out.println(d.getId_docente()
63                                 + " - " + d.getNombre());
64                         }
65                     }
66             }
67         } while (opcion != 0);
68     }
69 }
```




```
63         + " - " + d.getApellido()
64         + " - " + d.getTelefono()
65         + " - " + d.getFecha_contrato()
66         + " - " + d.getCorreo());
67     }
68 }
69 break;
70
71 case 3:
72     System.out.print("ID del Docente a modificar: ");
73     String ide = sc.nextLine();
74
75     System.out.print("Nuevo nombre: ");
76     String nombree = sc.nextLine();
77
78     System.out.print("Nuevo apellido: ");
79     String apellidoe = sc.nextLine();
80
81     System.out.print("Nuevo Telefono: ");
82     String telefonoe = sc.nextLine();
83
84     System.out.print("Nuevo Correo: ");
85     String correoe = sc.nextLine();
86
87     if (controller.actualizar(ide, nombree, apellidoe, telefonoe, correoe)) {
88         System.out.println(" Docente actualizado");
89     } else {
90         System.out.println(" Error al actualizar");
91     }
92     break;
93
94     } else {
95         System.out.println(" Error al actualizar");
96     }
97     break;
98
99 case 4:
100     System.out.print("ID del Docente a eliminar: ");
101     String id = sc.nextLine();
102     if (controller.eliminar(id)) {
103         System.out.println(" Docente eliminado");
104     } else {
105         System.out.println(" Error al eliminar");
106     }
107     break;
108
109 case 0:
110     System.out.println(" Volviendo al menu principal...");
111     break;
112
113 default:
114     System.out.println(" Opcion no valida");
115     break;
116 }
117 } while (opcion != 0);
118 }
```

V.matricula

```
1 |
2 | package Vista;
3 | import java.util.List;
4 | import java.util.Scanner;
5 | import Controlador.MatriculaControlador;
6 | import Modelo.Matriculas;
7 | public class VistaMatricula {
8 |     public void MenuMatricula() {
9 |         Scanner sc = new Scanner(System.in);
10 |         MatriculaControlador controller = new MatriculaControlador();
11 |         int opcion;
12 |
13 |         do {
14 |             System.out.println("\n--- MENU DE ESTUDIANTES");
15 |             System.out.println("1. Registrar Matricula");
16 |             System.out.println("2. Mostrar Matriculas");
17 |             System.out.println("3. Actualizar Matricula");
18 |             System.out.println("4. Eliminar Matriculas");
19 |             System.out.println("0. Salir ");
20 |             System.out.print("Opcion: ");
21 |             opcion = sc.nextInt();
22 |             sc.nextLine(); // Limpiar buffer
23 |
24 |             switch (opcion) {
25 |                 case 1:
26 |                     System.out.print("Id Matricula: ");
27 |                     int id_matricula = sc.nextInt();
28 |
29 |                     System.out.print("Id estudiante: ");
30 |                     String id_estudiante = sc.nextLine();
31 |
32 |                     System.out.print("Id grupo: ");
33 |                     int id_grupo = sc.nextInt();
34 |
35 |                     System.out.print("Fecha de matricula YYYY-MM-DD: ");
36 |                     String fecha_matricula = sc.nextLine();
37 |
38 |                     System.out.print("Periodo academico: ");
39 |                     String periodo_academico = sc.nextLine();
40 |
41 |                     System.out.print("Estado de la matricula: ");
42 |                     String estado_matricula = sc.nextLine();
43 |
44 |
45 |                     if (controller.guardar( id_matricula, id_estudiante, id_grupo, fecha_matricula, periodo_academico, estado_matricula )) {
46 |                         System.out.println("✓ Estudiante guardado");
47 |                     } else {
48 |                         System.out.println("X Error al guardar");
49 |                     }
50 |                     break;
51 |
52 |                 case 2:
53 |                     List<Matriculas> lista = controller.obtener();
54 |
55 |                     System.out.println("\n--- LISTA DE MATRICULA");
56 |                     if (lista.isEmpty()) {
57 |                         System.out.println(" No hay matriculas registrados");
58 |                     } else {
59 |                         for (Matriculas m : lista) {
60 |                             System.out.println(m.getId_estudiante()
61 |                                 + " - " + m.getId_matricula()
62 |                                 + " - " + m.getId_grupo());
63 |                         }
64 |                     }
65 |                 }
66 |             } while (opcion != 0);
67 |     }
68 | }
```



```
64         + " - " + m.getId_grupo()
65         + " - " + m.getFecha_matricula()
66         + " - " + m.getPeriodo_academico()
67         + " - " + m.getEstado_matricula()
68         );
69     }
70 }
71 break;
72
73 case 3:
74     System.out.print("ID de la matricula modificar: ");
75     int ide = sc.nextInt();
76
77     System.out.print("Id del estudiante: ");
78     String id_estudiantem = sc.nextLine();
79
80     System.out.print("Id del grupo: ");
81     int id_grupom = sc.nextInt();
82
83     System.out.print("Periodo academico: ");
84     String periodo_academicom = sc.nextLine();
85
86     System.out.print("Estado de la matricula: ");
87     String estado_matriculam = sc.nextLine();
88
89     if (controller.actualizar(ide, id_estudiantem, id_grupom, periodo_academicom, estado_matriculam)) {
90         System.out.println("Matricula actualizado");
91     } else {
92         System.out.println(" Error al actualizar");
93     }
94     break;
```

```
95
96 case 4:
97     System.out.print("ID del Matricula a eliminar: ");
98     String id = sc.nextLine(); // CORREGIDO: int -> String
99     if (controller.eliminar(id)) {
100         System.out.println(" Matricula eliminado");
101     } else {
102         System.out.println(" Error al eliminar");
103     }
104     break;
105
106 case 0:
107     System.out.println(" Volviendo al menu principal...");
108     break;
109
110 default:
111     System.out.println(" Opcion no valida");
112     break;
113 }
114
115 } while (opcion != 0);
116 }
117 }
118 }
```

V.Asistencia

```
1 |
2 | package Vista;
3 |
4 | import java.util.List;
5 | import java.util.Scanner;
6 | import Controlador.AsistenciaControlador;
7 | import Modelo.Asistencia;
8 |
9 | public class VistaAsistencia {
10 |     public void MenuAsistencia() {
11 |         Scanner sc = new Scanner(System.in);
12 |         AsistenciaControlador controller = new AsistenciaControlador();
13 |         int opcion;
14 |
15 |         do {
16 |             System.out.println("\n--- MENU DE ESTUDIANTES");
17 |             System.out.println("1. Registrar asistencia");
18 |             System.out.println("2. Mostrar asistencia");
19 |             System.out.println("3. Actualizar asistencia");
20 |             System.out.println("4. Eliminar asistencia");
21 |             System.out.println("0. Salir ");
22 |             System.out.print("Opcion: ");
23 |             opcion = sc.nextInt();
24 |             sc.nextLine(); // Limpiar buffer
25 |
26 |             switch (opcion) {
27 |                 case 1:
28 |                     System.out.print("Id asistencia: ");
29 |                     int id_asistencia = sc.nextInt();
30 |
31 |                     System.out.print("Id matricula: ");
32 |
33 |                     int id_matricula = sc.nextInt();
34 |
35 |                     System.out.print("Id grupo asignatura: ");
36 |                     int id_grupo_asign = sc.nextInt();
37 |
38 |                     System.out.println("Estado del estudiante: ");
39 |                     String estado = sc.nextLine();
40 |                     sc.nextLine();
41 |
42 |                     System.out.print("Fecha YYYY-MM-DD: ");
43 |                     String fecha = sc.nextLine();
44 |
45 |
46 |                     System.out.print("Observacion: ");
47 |                     String observacion = sc.nextLine();
48 |
49 |
50 |
51 |                     if (controller.guardar(id_asistencia, id_matricula, id_grupo_asign, fecha, estado, observacion)) {
52 |                         System.out.println("✓ Estudiante guardado");
53 |                     } else {
54 |                         System.out.println("✗ Error al guardar");
55 |                     }
56 |                     break;
57 |
58 |                 case 2:
59 |                     List<Asistencia> lista = controller.obtener();
60 |
61 |                     System.out.println("\n--- LISTA DE ASISTENCIA");
62 |                     if (lista.isEmpty()) {
```



```
63         System.out.println(" No hay asistencias registrados");
64     } else {
65         for (Asistencia a : lista) {
66             System.out.println(a.getId_asistencia()
67                 + " - " + a.getId_matricula()
68                 + " - " + a.getId_grupo_asign()
69                 + " - " + a.getFecha()
70                 + " - " + a.getEstado()
71                 + " - " + a.getObservacion()
72             );
73         }
74     }
75 }
76 break;
77
78 case 3:
79     System.out.print("ID de la asistencia modificar: ");
80     int ide = sc.nextInt();
81     sc.nextLine();
82
83     System.out.print("Fecha: ");
84     String fecham = sc.nextLine();
85
86     System.out.print("Estado: ");
87     String estadom = sc.nextLine();
88
89     System.out.print("Observacion: ");
90     String observacionm = sc.nextLine();
91
92
93
94     if (controller.actualizar(ide, fecham, estadom, observacionm)) {
95         System.out.println("asistencia actualizado");
96     } else {
97         System.out.println(" Error al actualizar");
98     }
99     break;
100
101 case 4:
102     System.out.print("ID de la asistencia a eliminar: ");
103     String id = sc.nextLine(); // CORREGIDO: int -> String
104     if (controller.eliminar(id)) {
105         System.out.println(" Asistencia eliminado");
106     } else {
107         System.out.println(" Error al eliminar");
108     }
109     break;
110
111 case 0:
112     System.out.println(" Volviendo al menu principal...");
113     break;
114
115 default:
116     System.out.println(" Opcion no valida");
117     break;
118 }
119
120 } while (opcion != 0);
121 }
122 }
123
```

v.calificaciones

```
1 package Vista;
2
3 import java.util.List;
4 import java.util.Scanner;
5 import Controlador.CalificacionesControlador;
6 import Modelo.Calificaciones;
7
8 public class VistaCalificaciones {
9     public void MenuCalificaciones() {
10         Scanner sc = new Scanner(System.in);
11         CalificacionesControlador controller = new CalificacionesControlador();
12         int opcion;
13
14         do {
15             System.out.println("\n--- MENU DE ESTUDIANTES");
16             System.out.println("1. Registrar calificaciones");
17             System.out.println("2. Mostrar calificaciones");
18             System.out.println("3. Actualizar calificaciones");
19             System.out.println("4. Eliminar calificaciones");
20             System.out.println("0. Salir ");
21             System.out.print("Opcion: ");
22             opcion = sc.nextInt();
23             sc.nextLine(); // Limpiar buffer
24
25             switch (opcion) {
26                 case 1:
27                     System.out.print("Id calificacion: ");
28                     int id_calificacion = sc.nextInt();
29
30                     System.out.print("Id matricula: ");
31                     int id_matricula = sc.nextInt();
32
33                     System.out.print("Id grupo asignatura: ");
34                     int id_grupo_asign = sc.nextInt();
35
36
37                     System.out.println("Promedio: ");
38                     String promedio = sc.nextLine();
39                     sc.nextLine();
40
41                     System.out.print("Fecha_registro YYYY-MM-DD: ");
42                     String fecha_registro = sc.nextLine();
43
44
45                     if (controller.guardar(id_calificacion, id_matricula, id_grupo_asign, promedio, fecha_registro)) {
46                         System.out.println("✓ Estudiante guardado");
47                     } else {
48                         System.out.println("✗ Error al guardar");
49                     }
50                     break;
51
52                 case 2:
53                     List<Calificaciones> lista = controller.obtener();
54
55                     System.out.println("\n--- LISTA DE CALIFICACIONES");
56                     if (lista.isEmpty()) {
57                         System.out.println("No hay calificaciones registrados");
58                     } else {
59                         for (Calificaciones c : lista) {
60                             System.out.println(c.getId_calificacion()
61                                 + " - " + c.getId_matricula()
62                                 + " - " + c.getId_grupo_asign());
63                         }
64                     }
65             }
66         } while (opcion != 0);
67     }
68 }
```




```
63         + " - " + c.getPromedio()
64         + " - " + c.getFecha_registro()
65
66     );
67
68     }
69     break;
70
71     case 3:
72         System.out.print("ID de la calificacion a modificar: ");
73         int ide = sc.nextInt();
74         sc.nextLine();
75
76         System.out.print("Promedio: ");
77         String promedioc = sc.nextLine();
78
79         System.out.print("Estado: ");
80         String fecha_registroc = sc.nextLine();
81
82
83         if (controller.actualizar(ide, promedioc, fecha_registroc)) {
84             System.out.println("Calificaion actualizado");
85         } else {
86             System.out.println(" Error al actualizar");
87         }
88         break;
89
90     case 4:
91         System.out.print("ID de la calificacion a eliminar: ");
92         String id = sc.nextLine(); // CORREGIDO: int -> String
93         if (controller.eliminar(id)) {
94             System.out.println(" Calificacion eliminado");
95         } else {
96             System.out.println(" Error al eliminar");
97         }
98         break;
99
100    case 0:
101        System.out.println(" Volviendo al menu principal...");
102        break;
103
104    default:
105        System.out.println(" Opcion no valida");
106        break;
107    }
108
109    } while (opcion != 0);
110
111 }
112
113 }
114
```



Vista Main

```
1
2 package Vista;
3 import Conexion.conexion;
4 import java.util.Scanner;
5 import Conexion.conexion;
6
7 public class SistemaAcademico_ {
8
9
10     public static void main(String[] args) {
11         conexion.conectar();
12         Scanner sc = new Scanner(System.in);
13
14         int opcion;
15
16         do {
17             conexion cn = new conexion();
18             System.out.println("\n--- SISTEMA DE ESTUDIANTES");
19             System.out.println("1. Perfil Estudiante");
20             System.out.println("2. Perfil docente");
21             System.out.println("3. Matriculas");
22             System.out.println("4. Asistencia");
23             System.out.println("5. Calificaciones");
24             System.out.println("0. Salir ");
25             System.out.print("Opcion: ");
26             opcion = sc.nextInt();
27
28             switch (opcion) {
29                 case 1:
30                     VistaEstudiante ve= new VistaEstudiante();
31                     ve.MenuEstudiante();
```



```
32         break;
33         case 2:
34             VistaDocentes vd = new VistaDocentes();
35             vd.MenuDocente();
36             break;
37         case 3:
38             VistaMatricula vm = new VistaMatricula();
39             vm.MenuMatricula();
40             break;
41         case 4:
42             VistaAsistencia va = new VistaAsistencia();
43             va.MenuAsistencia();
44             break;
45         case 5 :
46             VistaCalificaciones vc = new VistaCalificaciones();
47             vc.MenuCalificaciones();
48             break;
49         case 0:
50             System.out.println("\n ;Hasta luego!");
51             break;
52
53         default:
54             System.out.println(" Opción no válida");
55             break;
56     }
57     break;
58
59     }while (opcion != 0);
60
61
62     sc.close();
```

Conexion A BD

```
1
2 package Conexion;
3
4 import java.sql.Connection;
5 import java.sql.DriverManager;
6
7 public class conexion {
8
9     private static final String URL
10         = "jdbc:mysql://127.0.0.1:3306/sistemaacademico";
11     private static final String USER = "root";
12     private static final String PASSWORD = "";
13
14     public static Connection conectar() {
15         Connection con = null;
16         try {
17             con = DriverManager.getConnection(
18                 URL,
19                 USER,
20                 PASSWORD
21             );
22             System.out.println("Conexion exitosa");
23         } catch (Exception e) {
24             System.out.println("Error" + e.getMessage());
25         }
26         return con;
27     }
28 }
29
```

Previsualización de paquetes

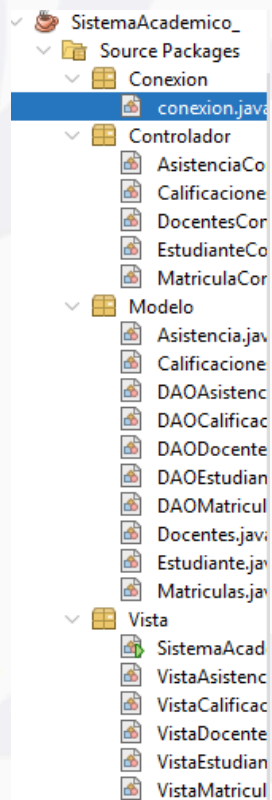
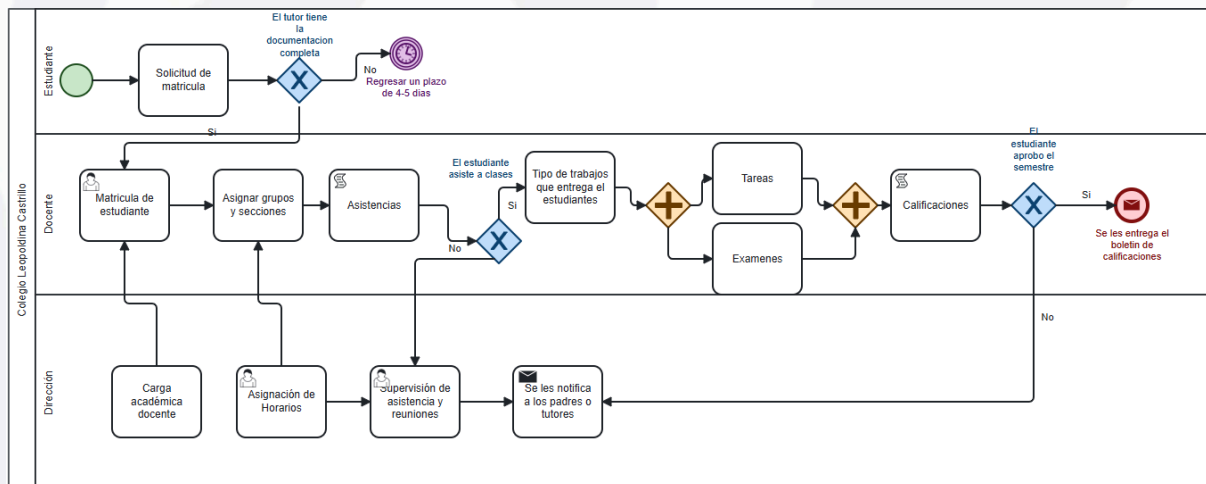


Diagrama BPMN



Instrumento de levantamiento de información

<https://forms.office.com/r/eWhYueggv7>



Análisis estadístico descriptivo de resultados

La gestión administrativa y académica en las escuelas siempre ha sido un proceso manual, lleno de errores y demoras. Pero una encuesta que hicimos a 40 docentes, estudiantes y personal administrativo muestra algo claro: ya urge un sistema académico digital que integre el control de asistencia y calificaciones.

El 95% de los encuestados dice que tener ese sistema es entre "importante" y "muy importante". Esto refleja que todos somos conscientes de lo mal que funciona lo tradicional. Lo que más valoran es poder organizar mejor el tiempo, equivocarse menos y tener la información segura. Cosas que, al final, mejoran la educación de verdad.

Entre los problemas más comunes que viven a diario están: calificaciones que se confunden, nombres mal escritos y notas que siempre llegan tarde. Todo eso pasa de "de vez en cuando" a "frecuentemente". Y aunque parezcan detalles pequeños, terminan generando desconfianza, desorden y mucho más trabajo para todos.

Algo curioso: el 75% cree que un sistema automático solucionaría estos problemas "en gran medida", y un 20% dice que "parcialmente". O sea, tienen mucha fe en la tecnología, pero sin pedir milagros.

Y lo que más piden son funciones claras: registro automático de calificaciones, control de asistencia, perfil del estudiante, gestión de matrículas e instrumentos de evaluación. Además, insisten en que el sistema se pueda usar desde cualquier dispositivo y que tenga buenos respaldos de seguridad. Nada del otro mundo, solo lo básico para que funcione bien.

Manual de usuario

El Sistema de Gestión Académica es una aplicación de escritorio que permite administrar los procesos académicos y administrativos del Centro de Estudio Leopoldina Castrillo.

Este manual guía a los usuarios en el uso del sistema, explicando cada funcionalidad disponible.

ACCESO AL SISTEMA

Para iniciar:

Ejecute el archivo SistemaAcademico.jar

Aparece el menú principal con las siguientes opciones:





Gestionar Estudiantes

Gestionar Docentes

Gestionar Matrículas

Gestionar Asistencias

Gestionar Calificaciones

Salir del sistema

Navegación:

Ingrese el número de la opción deseada y presione Enter

Seleccione la opción "Salir" para volver al menú anterior o cerrar el programa

MÓDULO DE ESTUDIANTES

Menú de Estudiantes:

Guardar estudiante

Mostrar estudiantes

Actualizar estudiante

Eliminar estudiante

Volver al menú principal

Guardar Estudiante:

Complete los siguientes campos:

ID Estudiante: Identificador único, por ejemplo: E001

Nombre: Nombre del estudiante

Apellido: Apellido del estudiante

Nombre del tutor: Nombre del padre, madre o tutor

Teléfono: Número de contacto





Cédula: Número de cédula de identidad

Fecha de nacimiento: Formato AAAA-MM-DD, por ejemplo: 2000-05-15

Estado: Activo, Inactivo, Graduado o Retirado

Mostrar Estudiantes:

Muestra la lista completa de estudiantes registrados con todos sus datos

Actualizar Estudiante:

Ingrese el ID del estudiante a modificar

Complete los campos que desea actualizar: nombre, apellido, estado o teléfono

Eliminar Estudiante:

Ingrese el ID del estudiante a eliminar

Confirme la operación

MÓDULO DE DOCENTES

Menú de Docentes:

Guardar docente

Mostrar docentes

Actualizar docente

Eliminar docente

Volver al menú principal

Guardar Docente:

Complete los siguientes campos:

ID Docente: Identificador único, por ejemplo: D001

Nombre: Nombre del docente

Apellido: Apellido del docente

Teléfono: Número de contacto





Fecha de contrato: Formato AAAA-MM-DD

Correo: Correo electrónico institucional

Mostrar Docentes:

Muestra la lista completa de docentes registrados

Actualizar Docente:

Ingrese el ID del docente a modificar

Complete los campos a actualizar: nombre, apellido, teléfono o correo

Eliminar Docente:

Ingrese el ID del docente a eliminar

Confirme la operación

MÓDULO DE MATRÍCULAS

Menú de Matrículas:

Registrar matrícula

Mostrar matrículas

Actualizar matrícula

Eliminar matrícula

Volver al menú principal

Registrar Matrícula:

Complete los siguientes campos:

ID Estudiante: Identificador del estudiante

ID Grupo: Identificador del grupo

Fecha de matrícula: Formato AAAA-MM-DD

Periodo académico: Por ejemplo: 2026-1





Estado de la matrícula: Activa o Cancelada

Mostrar Matrículas:

Muestra el listado completo de matrículas registradas

Actualizar Matrícula:

Ingrese el ID de la matrícula a modificar

Modifique los campos: estudiante, grupo, periodo o estado

Eliminar Matrícula:

Ingrese el ID de la matrícula a eliminar

Confirme la operación

MÓDULO DE ASISTENCIAS

Menú de Asistencias:

Registrar asistencia

Mostrar asistencias

Actualizar asistencia

Eliminar asistencia

Volver al menú principal

Registrar Asistencia:

Complete los siguientes campos:

ID Asistencia: Identificador único

ID Matrícula: Identificador de la matrícula

ID Grupo Asignatura: Identificador del grupo-asignatura

Fecha: Formato AAAA-MM-DD

Estado: Presente, Ausente, Tardanza o Justificado

Observación: Comentario adicional, opcional





Mostrar Asistencias:

Muestra el registro completo de asistencias

Actualizar Asistencia:

Ingrese el ID de la asistencia a modificar

Modifique: fecha, estado u observación

Eliminar Asistencia:

Ingrese el ID de la asistencia a eliminar

Confirme la operación

MÓDULO DE CALIFICACIONES

Menú de Calificaciones:

Registrar calificación

Mostrar calificaciones

Actualizar calificación

Eliminar calificación

Volver al menú principal

Registrar Calificación:

Complete los siguientes campos:

ID Calificación: Identificador único

ID Matrícula: Identificador de la matrícula

ID Grupo Asignatura: Identificador del grupo-asignatura

Grado: Nota numérica del cero al cien

Fecha de registro: Formato AAAA-MM-DD HH:MM

Mostrar Calificaciones:





Muestra el listado completo de calificaciones registradas

Actualizar Calificación:

Ingrese el ID de la calificación a modificar

Modifique: grado o fecha de registro

Eliminar Calificación:

Ingrese el ID de la calificación a eliminar

Confirme la operación

NAVEGACIÓN ENTRE MÓDULOS

Volver al Menú Principal:

Seleccione la opción "Salir" en cualquier módulo

El sistema regresa al menú principal automáticamente

Salir del Sistema:

En el menú principal, seleccione la opción "Salir"

El sistema se cierra completamente

ARQUITECTURA DEL SISTEMA

El sistema está desarrollado bajo el patrón Modelo-Vista-Controlador, conocido como MVC:

Modelo: Representa los datos y la lógica de negocio, incluye las clases: Estudiante, Docente, Matrícula, Asistencia y Calificación

Vista: Interfaz de usuario, incluye los menús y pantallas de cada módulo

Controlador: Intermediario entre el Modelo y la Vista, procesa las solicitudes del usuario

DAO (Data Access Object): Capa de acceso a datos que se conecta con la base de datos MySQL

SOLUCIÓN DE PROBLEMAS COMUNES

Error de conexión a base de datos:

Verifique que MySQL esté ejecutándose

Revise las credenciales de conexión en el código





Confirme que la base de datos sistemaacademico existe

Error al guardar fecha:

Utilice el formato AAAA-MM-DD, por ejemplo: 2026-05-17

No deje el campo de fecha vacío

Error al eliminar un registro:

El registro puede tener relaciones con otras tablas

Primero elimine los registros relacionados

El menú se cierra al volver:

Utilice siempre la opción "Salir" para volver al menú anterior

No utilice otras teclas para salir

GLOSARIO DE TÉRMINOS

Administrador: Usuario con permisos para gestionar estudiantes, docentes, matrículas y generar reportes

Asistencia: Registro de presencia o ausencia de un estudiante en clase

Calificación: Nota asignada para evaluar el rendimiento académico

CRUD: Crear, Leer, Actualizar, Eliminar, operaciones básicas de gestión de datos

Docente: Profesional que imparte clases y evalúa a los estudiantes

Estudiante: Persona matriculada en la institución educativa

Matrícula: Inscripción formal de un estudiante en un programa académico

MVC: Patrón Modelo-Vista-Controlador que organiza el software

JDBC: API de Java para conectar con bases de datos

MySQL: Sistema de gestión de bases de datos utilizado por la aplicación

FLUJO DE TRABAJO RECOMENDADO

Para Administradores:

Primero registre los docentes en el módulo de Docentes





Luego registre los estudiantes en el módulo de Estudiantes

Después realice las matrículas en el módulo de Matrículas

Finalmente registre asistencias y calificaciones en sus respectivos módulos

Para Docentes:

Registre las asistencias diarias de los estudiantes

Registre las calificaciones de evaluaciones y tareas

CONTACTO Y SOPORTE

Equipo de Desarrollo:

Dhiosmer José Cuadra Miranda

Soraya Lidieth Calero Calero

Ana Gabriela Irias Murillo

Institución:

Centro de Estudio Leopoldina Castrillo

Juigalpa, Chontales

Conclusión

El desarrollo del Sistema de Gestión Académica para el Centro de Estudio Leopoldina Castrillo ha permitido identificar y abordar las principales necesidades de la institución en materia de administración educativa. A continuación, se presentan las conclusiones más relevantes del proyecto:

El sistema implementado ha logrado digitalizar los procesos manuales que antes generaban pérdida de documentos, errores en los registros y demoras en la atención a padres y estudiantes. La aplicación de escritorio desarrollada permite almacenar de manera segura y organizada la información de estudiantes, docentes, matrículas, asistencias y calificaciones.

recomendacion

con base en los resultados obtenidos y las experiencias durante el desarrollo del proyecto, se formulan las siguientes recomendaciones para el Centro de Estudio Leopoldina Castrillo:





Capacitación del personal: Se recomienda capacitar al personal administrativo y docente en el uso del sistema para aprovechar al máximo sus funcionalidades y garantizar una transición exitosa del registro manual al digital.

Implementación gradual: Se sugiere realizar una implementación gradual del sistema, comenzando con los módulos de estudiantes y docentes, para luego incorporar matrículas, asistencias y calificaciones, permitiendo a los usuarios familiarizarse progresivamente con la herramienta.

Respaldo de datos: Es fundamental establecer una política de respaldo periódico de la base de datos para prevenir la pérdida de información ante fallos técnicos o incidentes.

Mantenimiento y actualización: Se recomienda mantener el sistema actualizado y realizar revisiones periódicas para corregir posibles errores y agregar nuevas funcionalidades según las necesidades de la institución.

Referencia Bibliográficas

- Alvarez, A. (2015). Subsistema de formación de educadores en servicio: lineamientos para la formación en el contexto de la evaluación docente. Logo Pedagogía.
- Banco Mundial. (2018). Informe sobre el desarrollo 2018: Aprender para hacer realidad la promesa de la educación (Licencia Creative Commons CC BY 3.0 IGO).
- Chiavenato, I. (2002). Administración en los nuevos tiempos. McGraw-Hill Interamericana.
- García, R. (2010, julio). La educación como sistema social complejo: una aproximación metodológica para trabajar en centros educativos en entornos sociales problemáticos con fracaso escolar. Temas para la educación, (9), 1-10.
- Henao, H., & Pontín, M. (2005). La familia en situación de desplazamiento. RUT informa sobre desplazamiento forzado en Colombia.
- López, J. (2002). La educación como un sistema complejo. Islas, 113-127.
- Martín, R. (2014). Contextos de aprendizaje: formales, no formales e informales. Ikastorratza, e-Revista de didáctica, (12), 1-14.



- Melgarejo, X. (2015). Gracias Finlandia. Qué podemos aprender del sistema educativo de más éxito (7ª ed.). Plataforma Editorial.
- Moratto, N., Zapata, J., & Messenger, T. (2015). Conceptualización del ciclo vital familiar: una mirada a la producción durante el periodo comprendido entre los años 2002 a 2015. CES Psicología, 8(2), 103-121.

Anexos

Cronograma de actividades I

Nº	Actividades/Tareas	Fecha de inicio	Fecha fin	Hrs Trabajo/responsable
1	Corrección y redacción	20/03/2026	01/04/2026	Dhiosmer Cuadra
2	Busqueda de informacion, indexadas	20/03/2026	04/04/2026	Ana Murillo
3	Introducción y desarrollo del tema	20/03/2026	01/04/2026	Soraya Calero
5	R Bibliográficas busqueda de informacion de matriculas, asistencia y calificaciones	01/04/2026	17/04/2026	Enger Gomez

Tabla de asistencia I

		Fecha / Hora:	
--	--	---------------	--



Nº	Nombres y apellidos	Aportes realizados por integrantes de equipo	Observaciones
1	Dhiosmer Jose Cuadra Miranda	Crear el diseño de figma y Redacción del documento, Descripción del tema y creación del github	Trabajo
2	Soraya Lidieth Calero Calero	Ayudó al nº 1 en figma y a la redacción del documento y descripción del tema y R bibliográficas	Trabajo
3	Ana Gabriela Irias Murillo	Ayudó en el documento la introducción y objetivo y específicos, referencias indexadas	No trabajo en todo el plazo del mes